

Chapter 2

Architectural Details and Assembly Language Programming of 8085 MPU

Credit: 10 hrs

Overview:

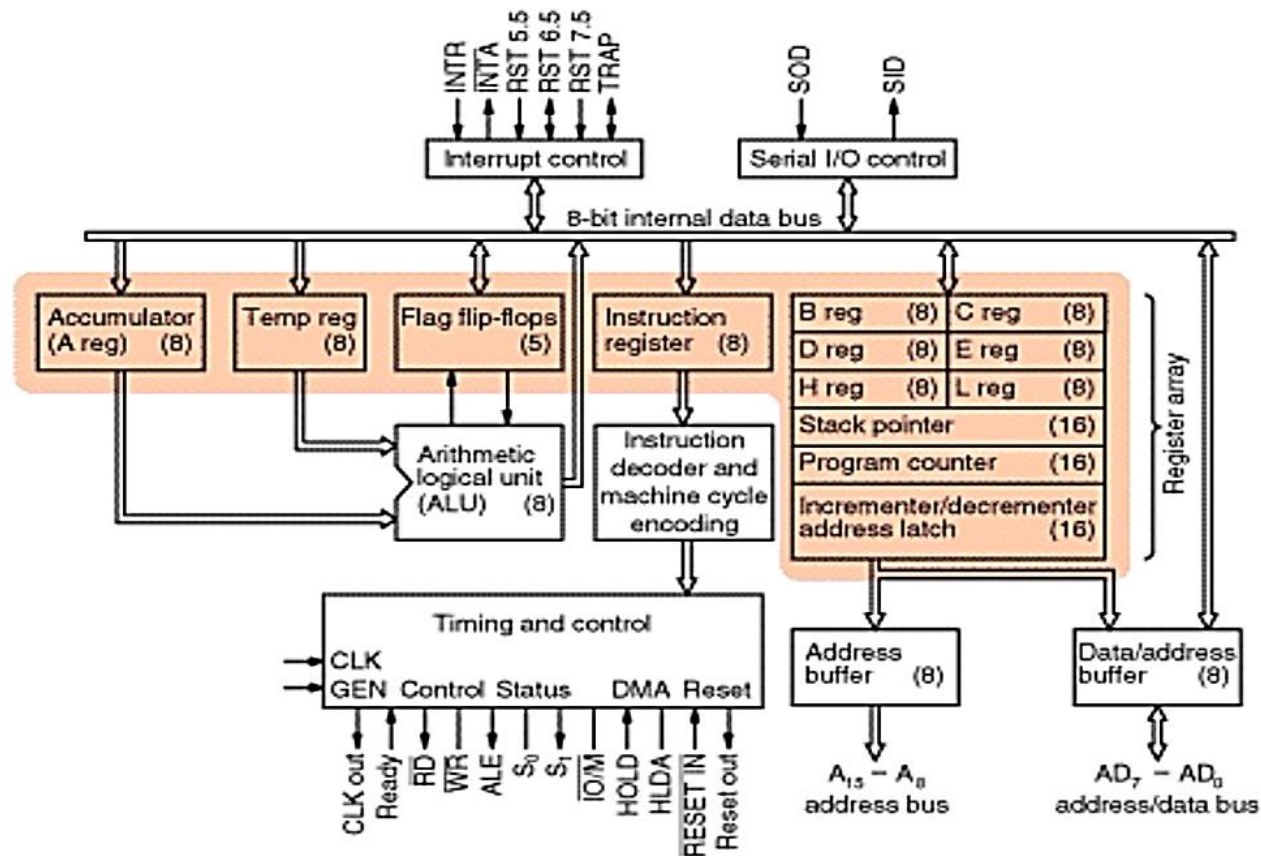
1. Internal architecture and description
2. Instruction classification
3. Addressing modes
4. Instruction Cycle, Machine cycle, t-state, Timing diagram, RTL
5. Basic Assembly language programs of 8085

1. Internal architecture and description

8085 Microprocessor: Introduction

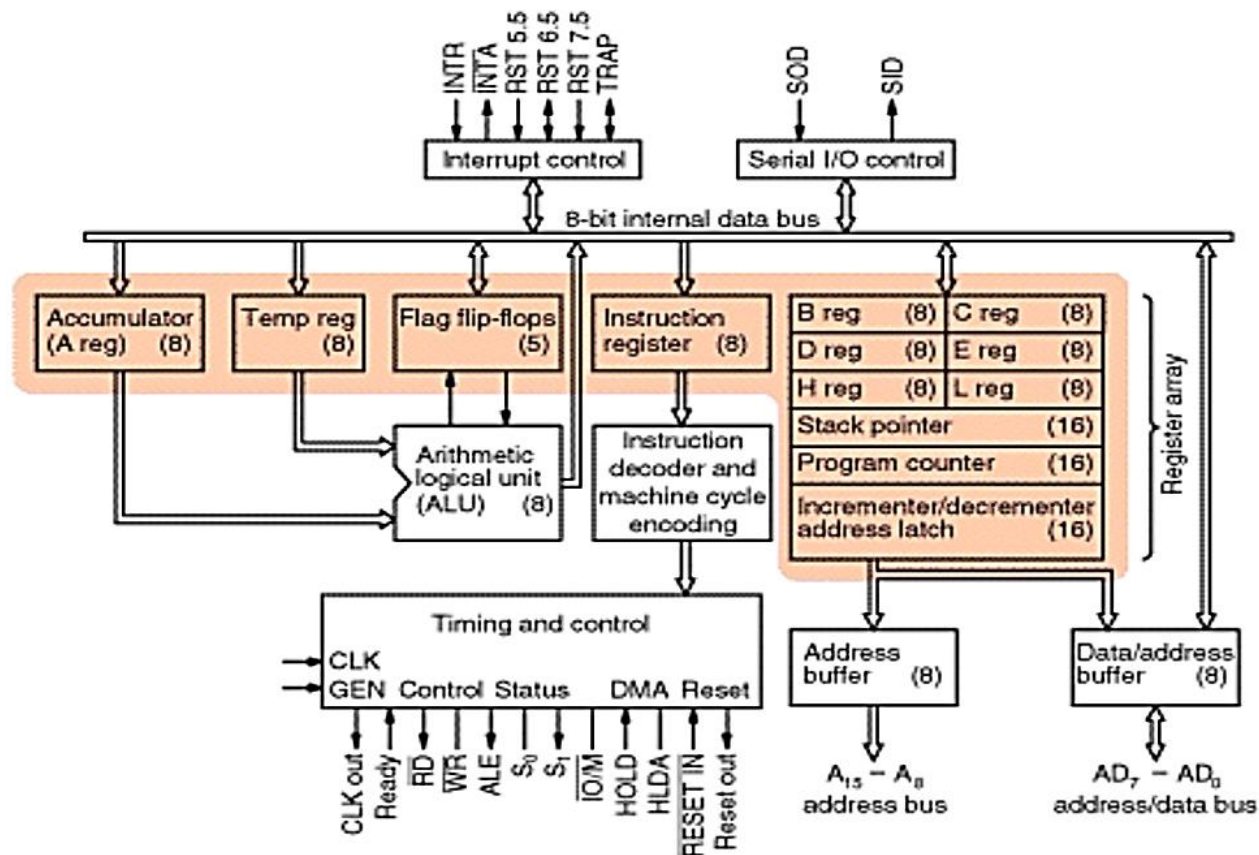
- 8085 is pronounced as "eighty-eighty-five" microprocessor. It is an 8-bit microprocessor designed by Intel in 1977 using NMOS technology.
- It has the following configuration –
 - 8-bit data bus
 - 16-bit address bus, which can address upto 64KB
 - A 16-bit program counter
 - A 16-bit stack pointer
 - Six 8-bit registers arranged in pairs: BC, DE, HL
 - Requires +5V supply to operate at 3.2 MHZ single phase clock
- It is used in washing machines, microwave ovens, mobile phones, etc

8085 MP- Block Diagram



8085 MP- functional Units

1. Accumulator
2. Arithmetic and logic unit
3. General purpose register
4. Program counter
5. Stack pointer
6. Flag register
7. Instruction register and decoder
8. Timing and control unit
9. Interrupt control
10. Serial Input/output control
11. Address buffer and address-data buffer
12. Address bus and data bus



8085 MP- functional Units

1. Accumulator

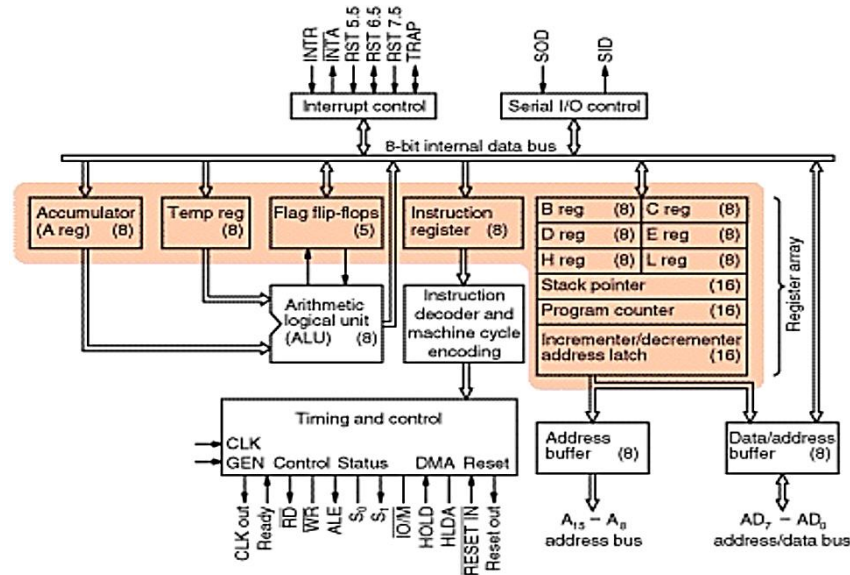
- It is an 8-bit register **used to store arithmetic, logical, I/O & LOAD/STORE operation results.**
- It is connected to internal data bus & ALU.

2. Arithmetic and logic unit

- It **performs arithmetic and logical operations** like Addition, Subtraction, AND, OR, etc. on 8-bit data.

3. General purpose register

- There are 6 general purpose registers in 8085 processor, i.e. B, C, D, E, H & L.
- Each register can **hold 8-bit data.**
- These registers **can work in pair to hold 16-bit data** and their pairing combination is like B-C, D-E & H-L.



8085 MP- functional Units

3. Program counter (PC)

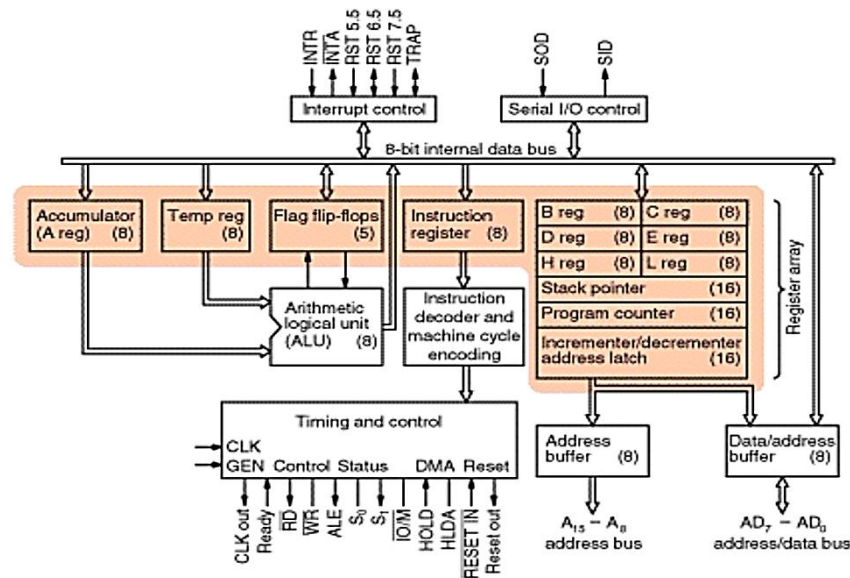
- It is a 16-bit register used to store the memory address location of the next instruction to be executed.
- The PC is incremented after each instruction is executed.
- Then, the PC points to the memory address of the next instruction that is going to be executed.

4. Stack pointer

- It is also a 16-bit register works like stack.
- It keeps track of the TOS (top of stack).
- It is always incremented/decremented by 2 during push & pop operations.

5. Temporary register

- It is an 8-bit register, which holds the temporary data of arithmetic and logical operations.



8085 MP- functional Units

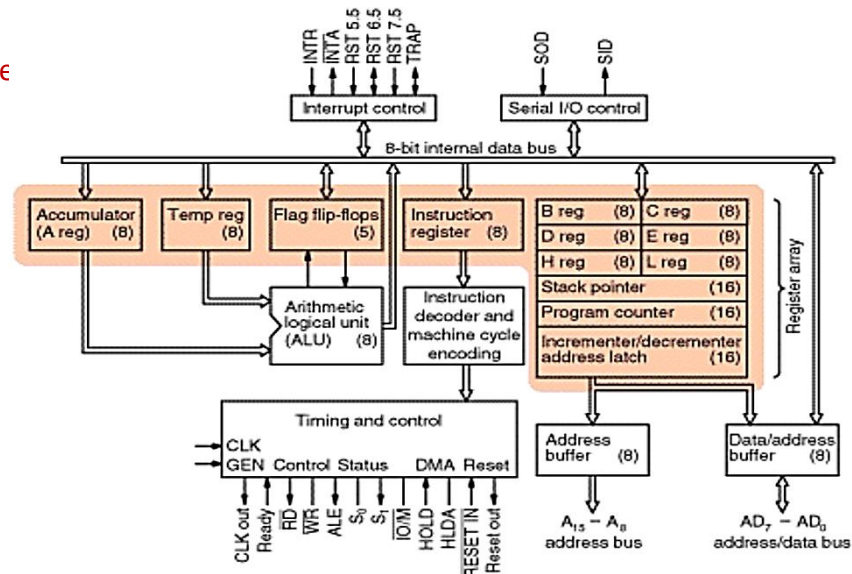
6. Flag register

- It is an 8-bit register having five 1-bit flip-flops, which holds either 0 or 1 depending upon the result stored in the accumulator.
- These are the set of 5 flip-flops –
 - Sign (S)
 - Zero (Z)
 - Auxiliary Carry (AC)
 - Parity (P)
 - Carry
- Its bit position is shown in the following table :

D7	D6	D5	D4	D3	D2	D1	D0
S	Z		AC		P		CY

7. Instruction Register and Decoder

- It stores the instruction fetched from the memory.
- Instruction decoder decodes the information present in the IR.



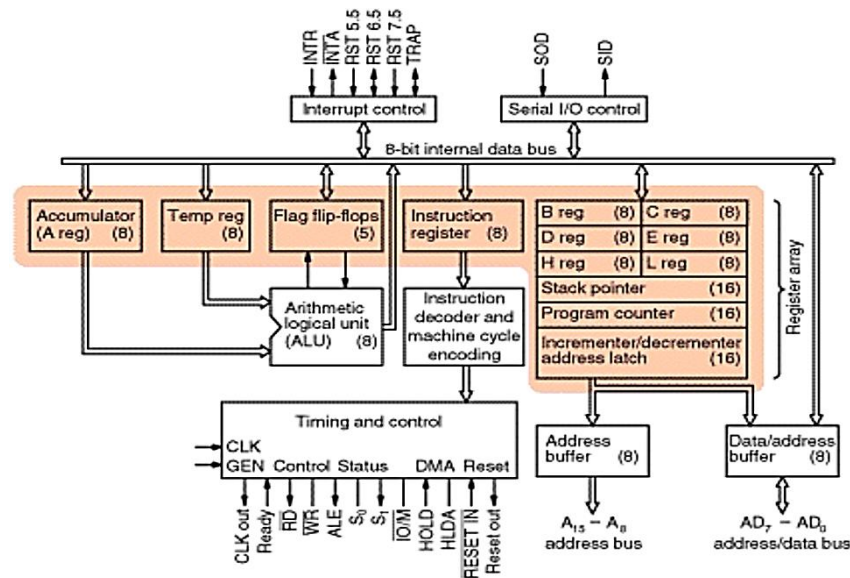
8085 MP- functional Units

8. Timing and control unit

- It provides timing and control signal to the microprocessor to perform operations.
- Following are the timing and control signals, which control external and internal circuits –
 - Control Signals:** READY, RD', WR', ALE
 - Status Signals:** S0, S1, IO/M'
 - DMA Signals:** HOLD, HLDA
 - RESET Signals:** RESET IN, RESET OUT

9. Interrupt control

- It controls the interrupts during a process.
- When a microprocessor is executing a main program and whenever an interrupt occurs, the microprocessor shifts the control from the main program to process the incoming request.
- After the request is completed, the control goes back to the main program.
- There are 5 interrupt signals in 8085 microprocessor: **INTR, RST 7.5, RST 6.5, RST 5.5, TRAP.**



8085 MP- functional Units

10. Serial Input/output control

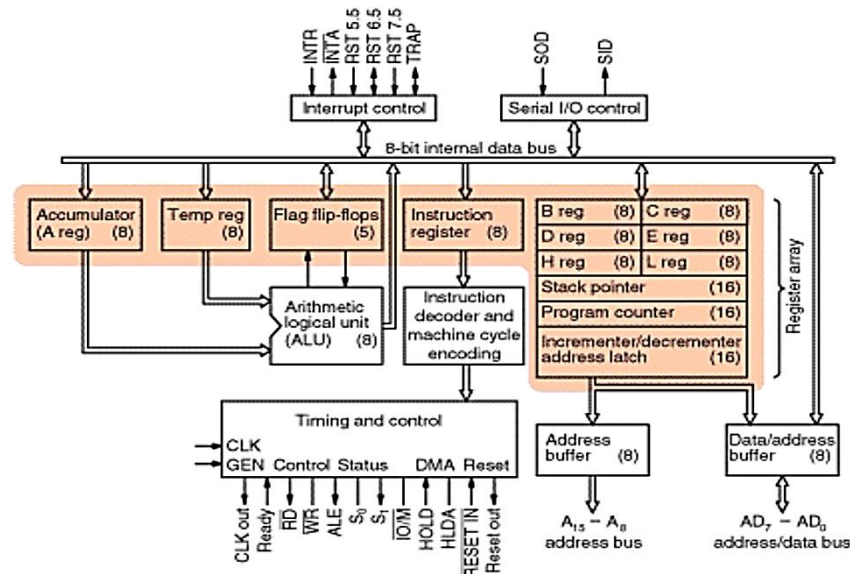
- It **controls the serial data communication** by using these two instructions: SID (Serial input data) and SOD (Serial output data).

11. Address buffer and address-data buffer

- The **content stored in the stack pointer and PC is loaded into the address buffer and address-data buffer** to communicate with the CPU.
- The memory and I/O chips are connected to these buses; the CPU can exchange the desired data with the memory and I/O chips.

12. Address bus and data bus

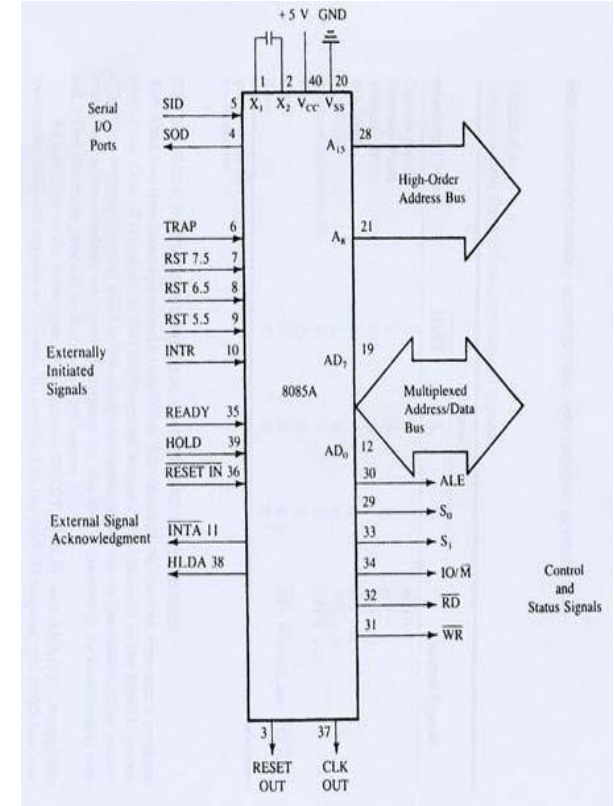
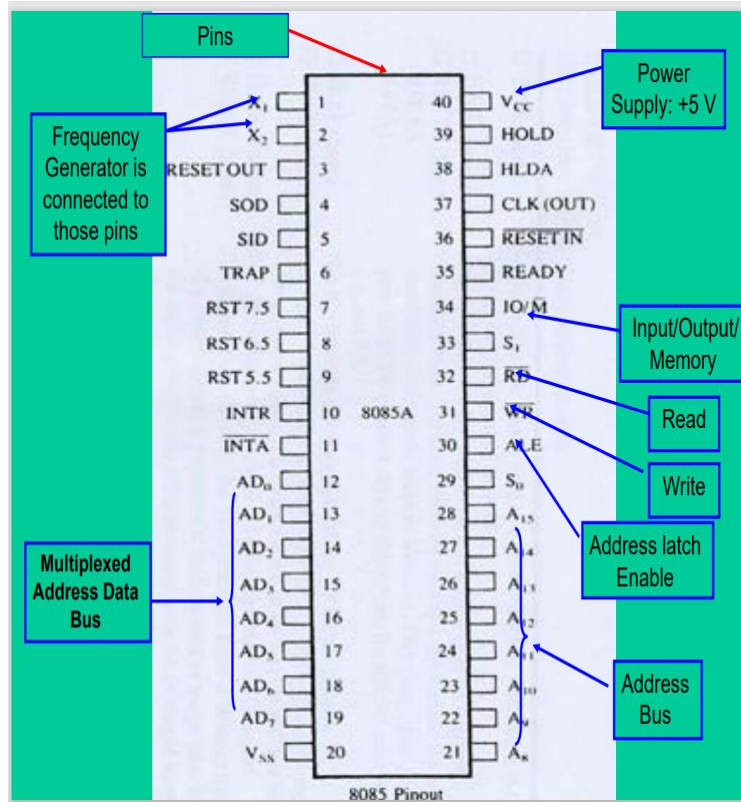
- Data bus **carries the data** to be stored. It is bidirectional.
- Address bus **carries the location (address)** to where it should be stored and it is unidirectional.



8085 Pin Configuration

The pins of a 8085 microprocessor can be classified into seven groups –

1. Address Bus
2. Data Bus
3. Control and Status Signals
4. Power Supply
5. Clock Signals
6. Interrupts & Externally initiated signals
7. Serial I/O signals



8085 Pin Configuration

1. Address bus

- **A15-A8**, it carries the most significant 8-bits of memory/IO address.

2. Data bus

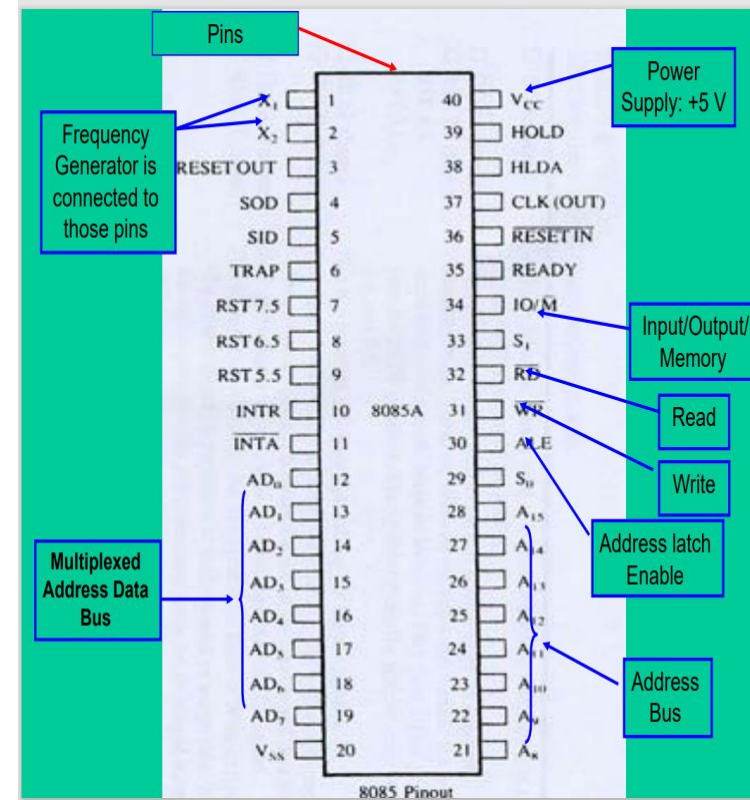
- **AD7-AD0**, it carries the least significant 8-bit address and data bus.

3. Power supply

- There are 2 power supply signals – VCC & VSS.
- **VCC** indicates +5v power supply and
- **VSS** indicates ground signal.

4. Clock signals

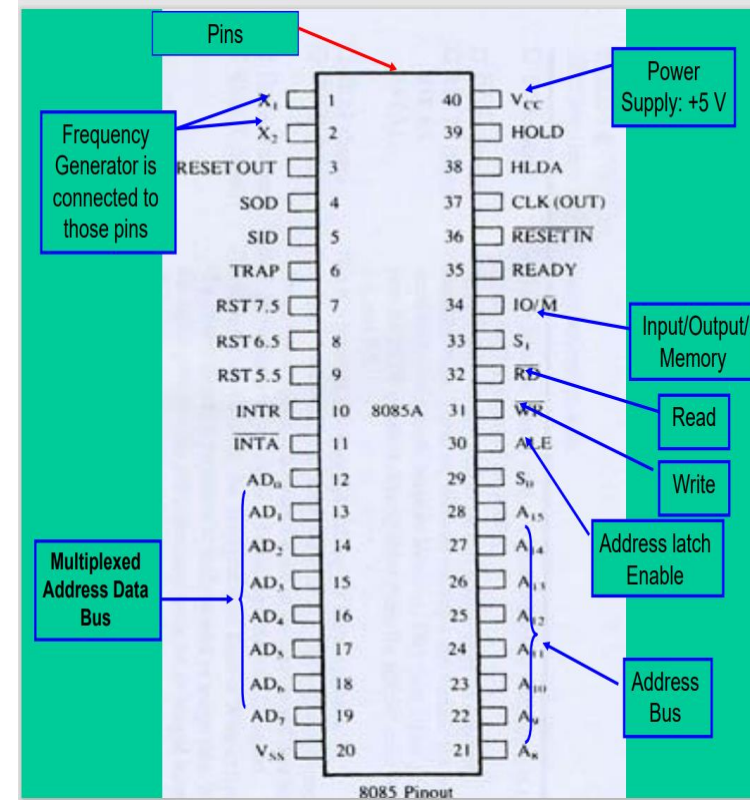
- There are 3 clock signals, i.e. X1, X2, CLK OUT.
 - **X1, X2** – A crystal (RC, LC N/W) is connected at these two pins and is used to set frequency of the internal clock generator. This frequency is internally divided by 2.
 - **CLK OUT** – This signal is used as the system clock for devices connected with the microprocessor



8085 Pin Configuration

5. Control and status signals

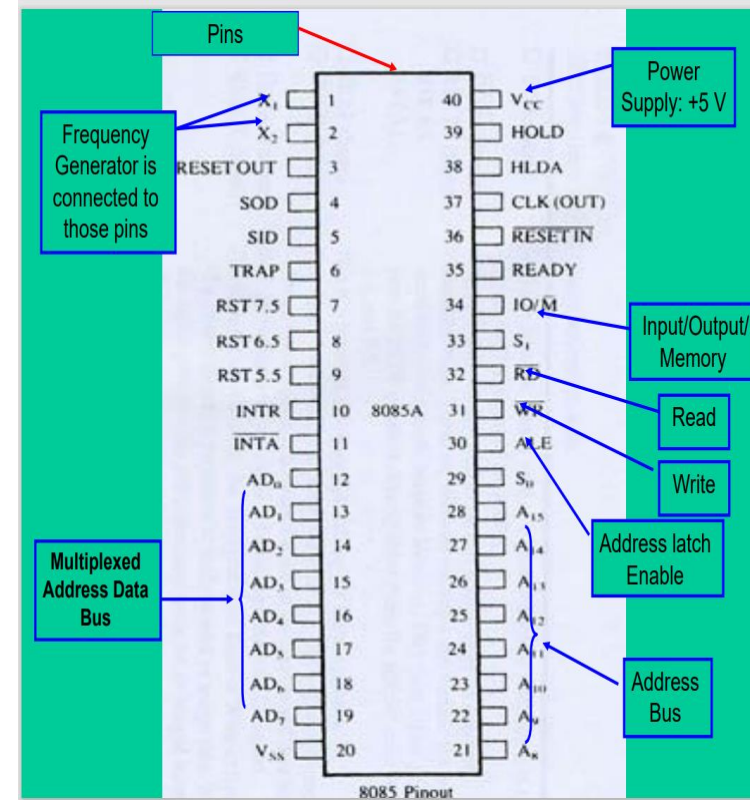
- These signals are **used to identify the nature of operation**.
- There are 3 control signal and 3 status signals.
- Three control signals are RD', WR & ALE.
 - RD'** – This signal indicates that the **selected IO or memory device** is to be read and is **ready for accepting data** available on the data bus.
 - WR'** – This signal indicates that the **data on the data bus is to be written** into a selected memory or IO location.
 - ALE** – It is a positive going pulse generated when a new operation is started by the microprocessor. When the **pulse goes high**, it **indicates address**. When the **pulse goes, down it indicates data**.
- Three status signals are IO/M', S0 & S1.
 - IO/M'**: This signal is used to differentiate between IO and Memory operations, i.e.
 - when it is **high indicates IO operation** and
 - when it is **low** then it **indicates memory operation**.
 - S1 & S0**: These signals are used to identify the type of current operation.



8085 Pin Configuration

6. Interrupts & externally initiated signals

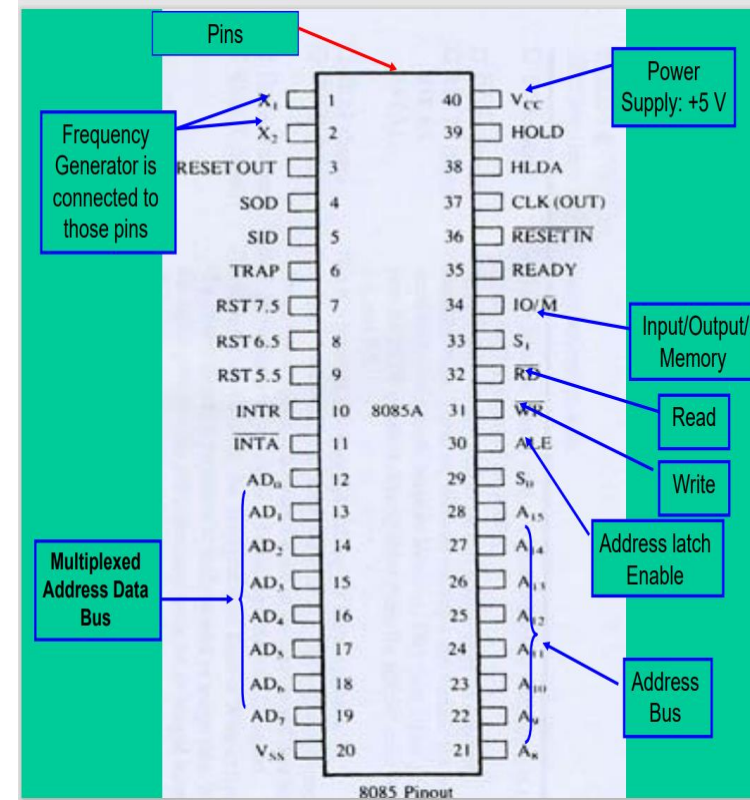
- Interrupts are the signals **generated by external devices** to request the microprocessor to perform a task.
- There are 5 interrupt signals, i.e. TRAP, RST 7.5, RST 6.5, RST 5.5, and INTR.
 1. **INTA** – It is an **interrupt acknowledgment** signal.
 2. **RESET IN** – This signal is **used to reset the microprocessor** by setting the PC to zero.
 3. **RESET OUT** – This signal is **used to reset all the connected devices** when the microprocessor is reset.
 4. **READY** – This signal **indicates that the device is ready to send or receive data**. If READY is low, then the CPU has to wait for READY to go high.
 5. **HOLD** – This signal **indicates that another master is requesting the use of the address and data buses**.
 6. **HLDA (HOLD Acknowledge)** – It **indicates** that the **CPU has received the HOLD request** and it will relinquish the bus in the next clock cycle. HLDA is set to low after the HOLD signal is removed.



8085 Pin Configuration

7. Serial I/O signals

- There are 2 serial signals, i.e. SID and SOD and these signals are **used for serial communication**.
 - **SOD** (Serial output data line) – The **output SOD is set/reset** as specified by the SIM instruction.
 - **SID** (Serial input data line) – The **data on this line is loaded into accumulator** whenever a RIM instruction is executed.



2. Instruction classification

Instruction classification

- An instruction is a binary pattern designed inside a microprocessor to perform a specific function.
- The **entire group of instructions is called instruction set.**
- Instruction set determines what functions the microprocessor can perform.
- Since the 8085 is an 8-bit device it can have up to 2^8 (256) instructions.
- However, the 8085 only uses 246 combinations that represent a total of 74 instructions.
- Most of the instructions have more than one format.
- Each instruction has two parts.
 - The first part is the task or operation to be performed.
 - This part is called the “**opcode**” (operation code).
 - The second part is the data to be operated on
 - Called the “**operand**”.

Instruction classification

- These instructions can be grouped into five different groups:
 1. Data Transfer Instructions
 2. Arithmetic Instructions
 3. Logic Instructions
 4. Branching Instructions
 5. Machine Control Instructions

1. Data Transfer Operations

- Data transfer instructions are the **instructions that transfer data** in the microprocessor.
- They are **also called copy instructions**.
- The data in the source is **not changed**.
- They transfer:
 - Data between registers.
 - Data Byte to a register or memory location.
 - Data between a memory location and a register.
 - Data between an I\O Device and the accumulator.
- Some of the Data transfer instruction are: MVI, LDA, STA,XCH, PUSH, POP,....

Opcode	Operand	Explanation of Instruction	Example
MOV	Rd, Rs M, Rs Rd, M	Copy from source(Rs) to destination(Rd)	MOV B, C or MOV B, M
MVI	Rd, data M, data	Move immediate 8-bit	MVI B, 57H or MVI M, 57H
LDA	16-bit address	Load accumulator	LDA 2034H
LDAX	B/D Reg. pair	Load accumulator indirect	LDAX B
LXI	Reg. pair, 16-bit data	Load register pair immediate	LXI H, 2034H
LHLD	16-bit address	Load H and L registers direct	LHLD 2040H
STA	16-bit address	16-bit address	STA 4350H
STAX	Reg. pair	Store accumulator indirect	STAX B
SHLD	16-bit address	Store H and L registers direct	SHLD 2470H
XCHG	none	Exchange H and L with D and E	XCHG
SPHL	none	Copy H and L registers to the stack pointer	SPHL
XTHL	none	Exchange H and L with top of stack	XTHL
PUSH	Reg. pair	Push register pair onto stack	PUSH B or PUSH A
POP	Reg. pair	Pop off stack to register pair	POP H or POP A
OUT	8-bit port address	Output data from accumulator to a port with 8-bit address	OUT F8H
IN	8-bit port address	Input data to accumulator from a port with 8-bit address	IN 8CH

2. Arithmetic Operations

- Data transfer instructions are the instructions that **perform arithmetic operations** such as addition, subtraction, increment and decrement.
- **Addition:** Any 8-bit number or contents of a register or contents of a memory location can be added to the contents of the accumulator and the **result is stored in the accumulator**.
- **Subtraction:** Any 8-bit number or contents of a register or contents of a memory location can be subtracted from the contents of the accumulator and the **result is stored in the accumulator**.
- **Increment/decrement:** The 8-bit contents of any memory location or any register can be directly incremented or decremented by 1. No need to disturb the contents of the accumulator.

Opcode	Operand	Meaning	Example
ADD	R M	Add register or memory, to the accumulator	ADD K
ADC	R M	Add register to the accumulator with carry	ADC K
ADI	8-bit data	Add the immediate to the accumulator	ADI 55K
ACI	8-bit data	Add the immediate to the accumulator with carry	ACI 55K
LXI	Reg. pair, 16bit data	Load the register pair immediate	LXI K, 3025M
DAD	Reg. pair	Add the register pair to H and L registers	DAD K
SUB	R M	Subtract the register or the memory from the accumulator	SUB K
SBB	R M	Subtract the source and borrow from the accumulator	SBB K
SUI	8-bit data	Subtract the immediate from the accumulator	SUI 55K
XCHG	None	Exchange H and L with D and E	XCHG
INR	R M	Increment the register or the memory by 1	INR K
INX	R	Increment register pair by 1	INX K
DCR	R M	Decrement the register or the memory by 1	DCR K
DCX	R	Decrement the register pair by 1	DCX K
DAA	None	Decimal adjust accumulator	DAA

3. Logical Operations

- These instructions **perform logic operations** on the contents of the accumulator.
 - ANA, ANI, ORA, ORI, XRA and XRI
 - Source: Accumulator and
 - An 8-bit number
 - The contents of a register
 - The contents of a memory location
 - Destination: Accumulator

ANA	R/M	AND Accumulator With Reg/Mem
ANI	#	AND Accumulator With an 8-bit number
ORA	R/M	OR Accumulator With Reg/Mem
ORI	#	OR Accumulator With an 8-bit number
XRA	R/M	XOR Accumulator With Reg/Mem
XRI	#	XOR Accumulator With an 8-bit number

4. Branching Operations

- This group of instructions **alters the sequence of program execution either conditionally or unconditionally.**
- The addresses supplied to all branch operations must be **16-bits.**
 - Unconditional branch.
 - Go to a new location no matter what.
 - Conditional branch.
 - Go to a new location if the condition is true

Unconditional Branch

- JMP Address
 - Jump to the address specified (Go to).
- CALL Address
 - Jump to the address specified but treat it as a subroutine.
- RET
 - Return from a subroutine.
- The addresses supplied to all branch operations must be **16-bits**.

Conditional Branch

- Go to new location if a specified condition is met.
 - JZ Address (Jump on Zero)
 - Go to address specified if the **Zero flag is set**.
 - JNZ Address (Jump on NOT Zero)
 - Go to address specified if the **Zero flag is not set**.
 - JC Address (Jump on Carry)
 - Go to the address specified if the **Carry flag is set**.
 - JNC Address (Jump on No Carry)
 - Go to the address specified if the **Carry flag is not set**.
 - JP Address (Jump on Plus)
 - Go to the address specified if the **Sign flag is not set**.
 - JM Address (Jump on Minus)
 - Go to the address specified if the **Sign flag is set**.

5. Machine control Operations

- These instructions control machine functions such as Halt, Interrupt or Do-nothing
 - HLT
 - Stop executing the program.
 - NOP
 - No operation
 - Exactly as it says, do nothing.
 - Usually used for delay or to replace instructions during debugging.

EI : Enable Interrupts

DI : Disable Interrupts

NOP : No operation

HLT : Halt the processor

SIM : Set Interrupt Mask

RID : Read Interrupt Mask

3. Addressing modes

Addressing modes in 8085

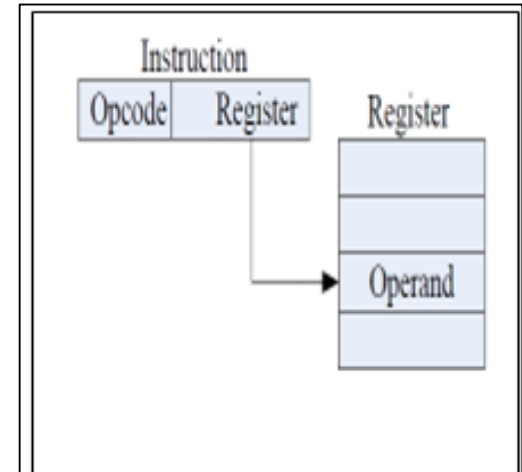
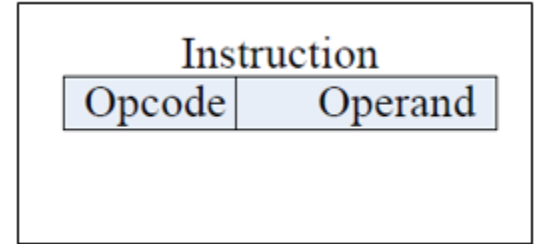
- The microprocessor has different ways of specifying the data for the instruction. These are called “addressing modes”.
- The 8085 has five addressing modes:
 1. Immediate Addressing mode
 2. Register Addressing mode
 3. Direct Addressing mode
 4. Indirect Addressing mode
 5. Implied Addressing mode

1. Immediate addressing mode

- In this mode, the 8/16-bit data is specified in the instruction itself as one of its operand.
- **For example:** MVI K, 20H
- means 20H is copied into register K.

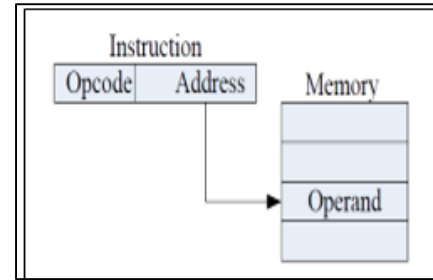
2. Register addressing mode

- In this mode, the data is copied from one register to another.
- **For example:** MOV K, B
- means data in register B is copied to register K.



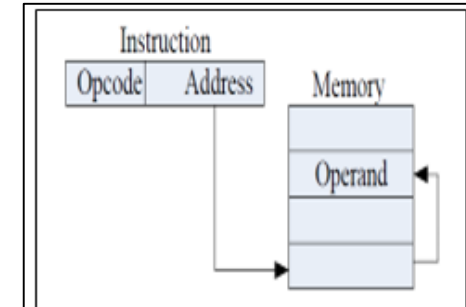
3. Direct addressing mode

- In this mode, the data is directly copied from the given address to the register.
- When instruction explicitly states the location of an operand or a destination (either in memory or in a processor register), the addressing mode is known as direct addressing.
- **For example:** LDB 5000K
- means the data at address 5000K is copied to register B.



4. Indirect addressing mode

- In this mode, the data is transferred from one register to another by using the address pointed by the register.
- **For example:** MOV K, B
- means data is transferred from the memory address pointed by the register to the register K.



5. Implied addressing mode

- This mode doesn't require any operand; the data is specified by the opcode itself.
- **For example:** CMP

4. Instruction Cycle, Machine cycle, t-state, Timing diagram, RTL

Instruction Cycle, Machine Cycle and T-state

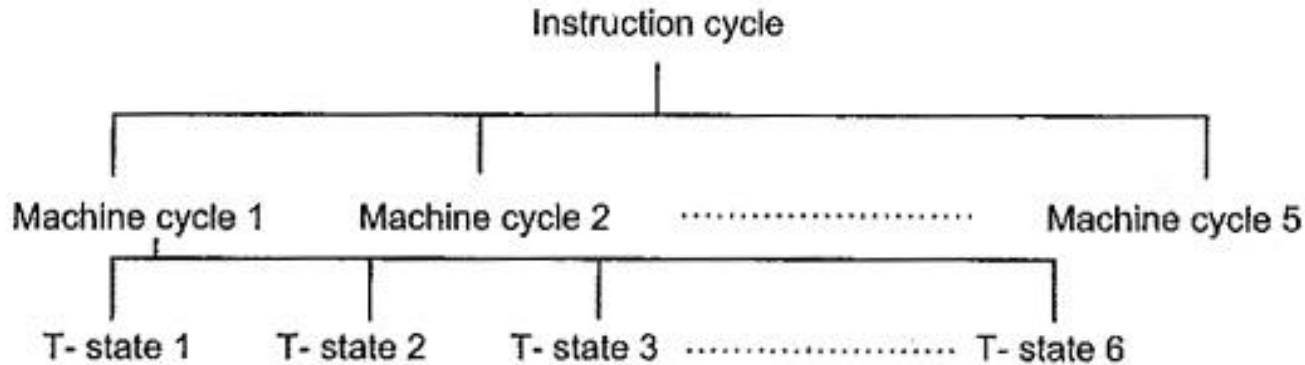


Fig. 1.4 Relation between instruction cycle, machine cycle and T-state

Instruction Cycle, Machine Cycle and T-state

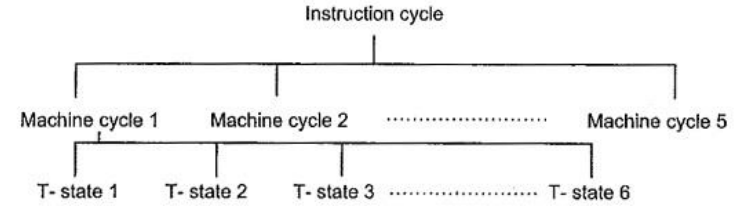
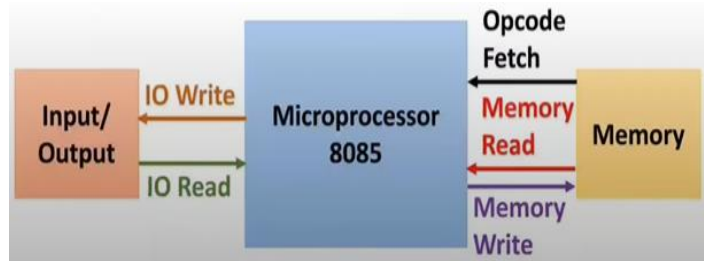


Fig. 1.4 Relation between instruction cycle, machine cycle and T-state

- During normal operation, the microprocessor sequentially **fetches, decodes and executes** one instruction after another until a halt instruction (HLT) is executed.
- The fetching, decoding and execution of a single instruction constitutes an **instruction cycle**, which consists of **one to six read or write operations** between processor and memory or input/output devices.

Instruction Cycle, Machine Cycle and T-state

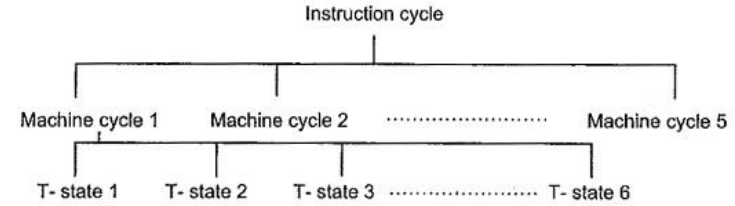
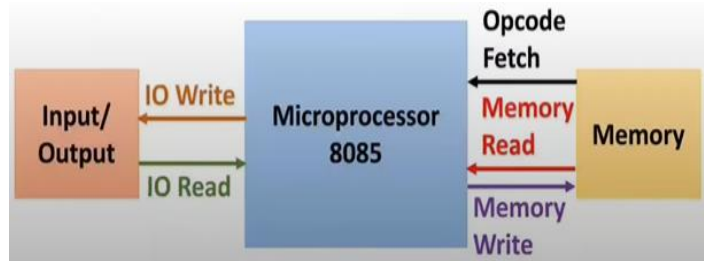
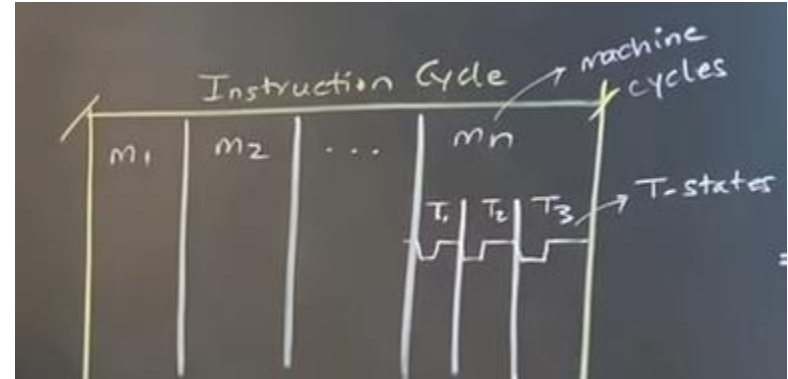
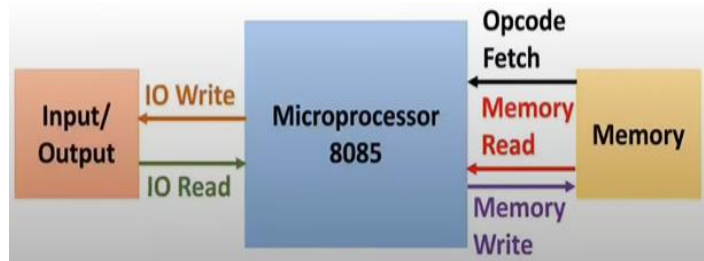


Fig. 1.4 Relation between instruction cycle, machine cycle and T-state

- Each memory or I/O operation requires a particular time period, called machine cycle.
- The time needed for completing one operation of accessing memory, I/O or acknowledging an external request is termed as **Machine cycle**.
- In other words, to move a byte of data in or out of the microprocessor, a machine cycle is required.
- To move 1 byte of data, 1 machine cycle is required. To move 2 byte of data, machine cycles are required.

Instruction Cycle, Machine Cycle and T-state



- Machine cycle is comprised of T-states.
- One subdivision of the operation completed in one clock period is termed as **T-state**.
- Each machine cycle consists of 3 to 6 clock periods/cycles, referred to as T-states.
- $T\text{-state} = \frac{1}{3} \text{ MHz} = 0.333 \text{ micro sec.}$

Reference: <https://www.youtube.com/watch?v=GoPh2PS25AU>

Architectural Details and Assembly
Language Programming of 8085 MPU

Timing Diagram

- Timing Diagram is a graphical representation of **execution time taken by each instruction** in a graphical format.
- The execution time is represented in T-states.

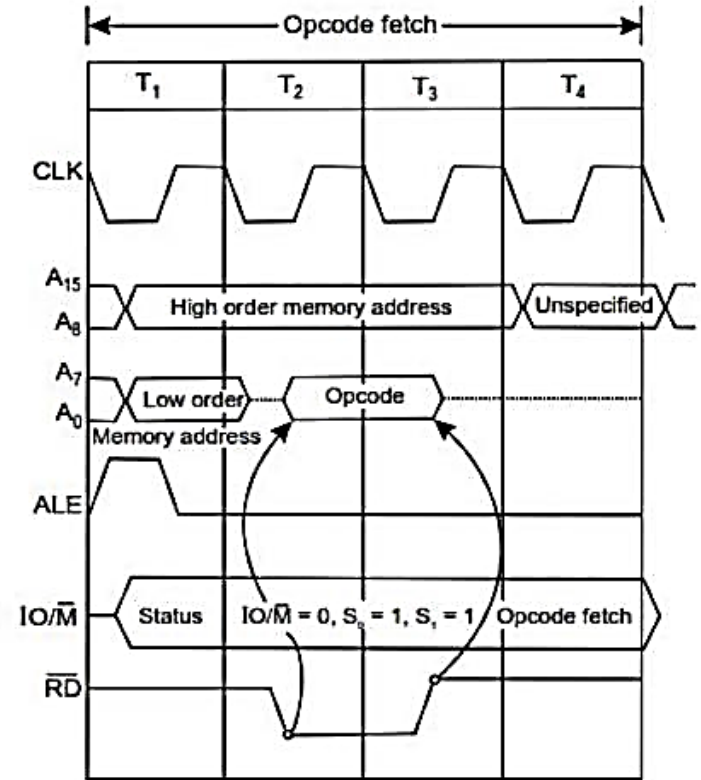


Fig 1.8 Opcode fetch machine cycle

Some Basics for Timing Diagram

Following buses and control signal must be shown in a timing diagram:

1. **Clock:** The time interval. Operates on a clock frequency of 3MHz- 6MHz
2. **A₈-A₁₅:** Higher Order Address
3. **AD₀-AD₇:** Lower Order Address
4. **ALE:** Look for ALE position.
If LOW, it carries data
If HIGH, it carries address
5. **$\overline{\text{RD}}$** : When LOW, it reads.
6. **$\overline{\text{WR}}$** : When LOW, write
7. **IO/ $\overline{\text{M}}$** : Operation either in Input-Output or Memory
8. **S0 and S1:** if 11, Opcode fetching operation, 01 and 10 for read or write respectively

Representation of 8085 signals

1. Clock Signal

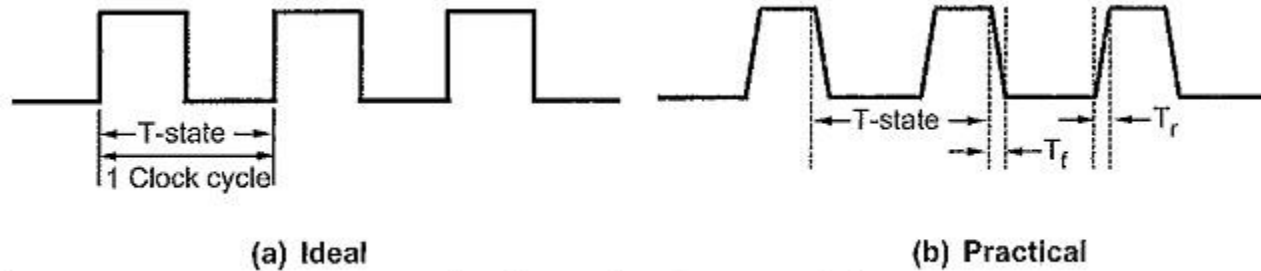


Fig. 1.5 Clock signal representation

2. Group of signals

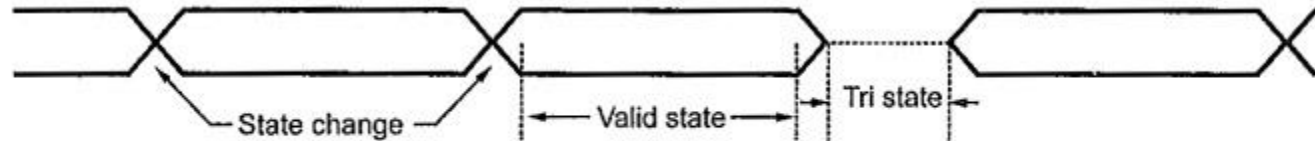


Fig. 1.7 Group of signals representation

Summary:

- **Instruction Cycle:**
 - The time required to execute an instruction is called instruction cycle.
- **Machine Cycle:**
 - The time required to access the memory or input/output devices is called machine cycle.
- **T-State:**
 - The machine cycle and instruction cycle takes multiple clock periods.
 - A portion of an operation carried out in one system clock period is called as T-state.
- **Timing diagram:**
 - Timing Diagram is a graphical representation of execution time taken by each instruction in a graphical format.

Instruction Cycle

- The execution of each instruction involves a series of steps known as the instruction cycle.
- The instruction cycle of the 8085 microprocessor consists of four basic steps, which are:
 1. **Fetch:** In this step, the microprocessor fetches the instruction from the memory location pointed to by the program counter (PC). The PC is incremented by one after the fetch operation.
 2. **Decode:** Once the instruction is fetched, the microprocessor decodes it to determine the operation to be performed and the operands involved.
 3. **Execute:** In this step, the microprocessor performs the operation specified by the instruction on the operands.
 4. **Store:** Finally, the result of the execution is stored in the appropriate memory location or register.
- Once the execution of an instruction is complete, the microprocessor returns to the fetch step to fetch the next instruction to be executed. This cycle repeats until the program is complete or interrupted

Machine Cycle

- The seven Machine Cycle in 8085 Microprocessor are :

1. **Opcode Fetch Cycle**
2. **Memory Read**
3. **Memory Write**
4. **I/O Read**
5. **I/O Write**
6. **Interrupt Acknowledge**
7. **Bus Idle**

Machine Cycle	Status			Control		
	$\overline{IO/\overline{M}}$	S_1	S_0	$\overline{RD}$	$\overline{WR}$	$\overline{INTA}$
Opcode Fetch	0	1	1	0	1	1
Memory Read	0	1	0	0	1	1
Memory Write	0	0	1	1	0	1
I/O Read	1	1	0	0	1	1
I/O Write	1	0	1	1	0	1
INTR Acknowledge	1	1	1	1	1	0
Bus Idle	0	0	0	1	1	1

Table 1.1 8085 machine cycles

- Three status signals $\overline{IO/\overline{M}}$, S_1 and S_0 identify each type as shown in Table 1.1.
- These signals are generated at the beginning of each machine cycle and remained valid for the duration, of the cycle.

Reference : <https://www.eeeguide.com/machine-cycle-in-8085-microprocessor/>

Machine cycle

- Machine cycle is defined as the **time required** for completing the operation of accessing either memory or I/O device.
- In the 8085, the machine cycle may consist of three to six T states.
- The T-state is defined as one **sub-division** of the operation performed in one clock period.
- These sub-divisions are internal states synchronized with the system clock.
- In every machine cycle the **first operation is op-code fetch** and the **remaining will be read or write from memory or IO devices.**

Machine Cycle

1: Opcode Fetch Cycle:

- It is the first Machine Cycle of every instruction in which the 8085 finds the nature of the instruction to be executed.
- The processor places the contents of the PC on the address lines, and through the read process, reads the opcode of the instruction.

2. Memory Read Cycle:

- The 8085 executes the memory read cycle to read the contents of R/W memory or ROM.

3. Memory Write Cycle:

- The 8085 executes the memory write cycle to store the data into data memory or stack memory.

4,5: I/O Read and I/O Write cycles:

- The I/O read and I/O write machine cycles are similar to the memory read and memory write machine cycles, respectively, except that the $\overline{IO}/M$ signal is high for I/O read and I/O write machine cycles.

6. Interrupt Acknowledge Cycle:

- In response to INTR signal, 8085 executes interrupt acknowledge machine cycle to read an instruction from the external device.

7. Bus Idle Cycle:

- There are few situations where the machine cycles are neither Read nor Write.

a) Opcode Fetch Machine cycle

- Each instruction of the processor has one byte opcode.
- The opcodes are stored in memory. So, the processor executes the opcode fetch machine cycle to fetch the opcode from memory.
- every instruction starts with opcode fetch machine cycle.
- The time taken by the processor to execute the opcode fetch cycle is 4T.
- In this time, the first, 3 T-states are used for fetching the opcode from memory and the remaining T-states are used for internal operations by the processor.

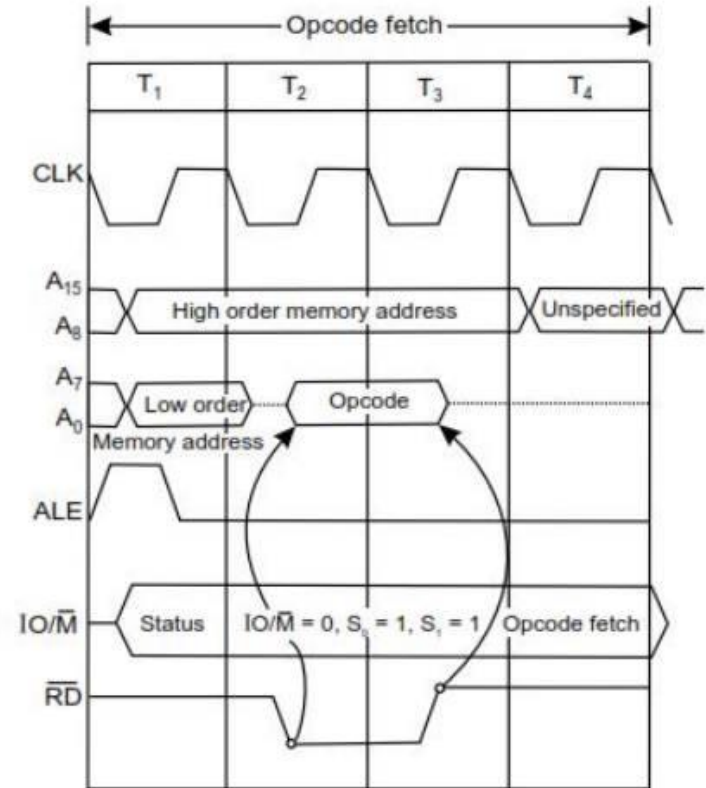


Fig 1.8 Opcode fetch machine cycle

The steps in Opcode Fetch machine cycle are

S. No	T state	Operation
1	T ₁	The microprocessor places the higher order 8-bits of the memory address on A ₁₅ – A ₈ address bus and the lower order 8-bits of the memory address on AD ₇ – A ₀ address / data bus.
2		The microprocessor makes the ALE signal HIGH and at the middle of T ₁ state, ALE signal goes LOW.
3		The status signals are changed as IO/ $\overline{M}$ ' = 0, S ₁ = 1 and S ₀ = 1. These status signals do not change throughout the OF machine cycle.
4	T ₂	The microprocessor makes the RD' line LOW to enable memory read and increments the Program Counter.
5		The contents on D ₇ – D ₀ (i.e. the Opcode) are placed on the address / data bus.
6	T ₃	The microprocessor transfers the Opcode on the address / data bus to Instruction Register (IR).
7		The microprocessor makes the RD' line HIGH to disable memory read.
8	T ₄	The microprocessor decodes the instruction.

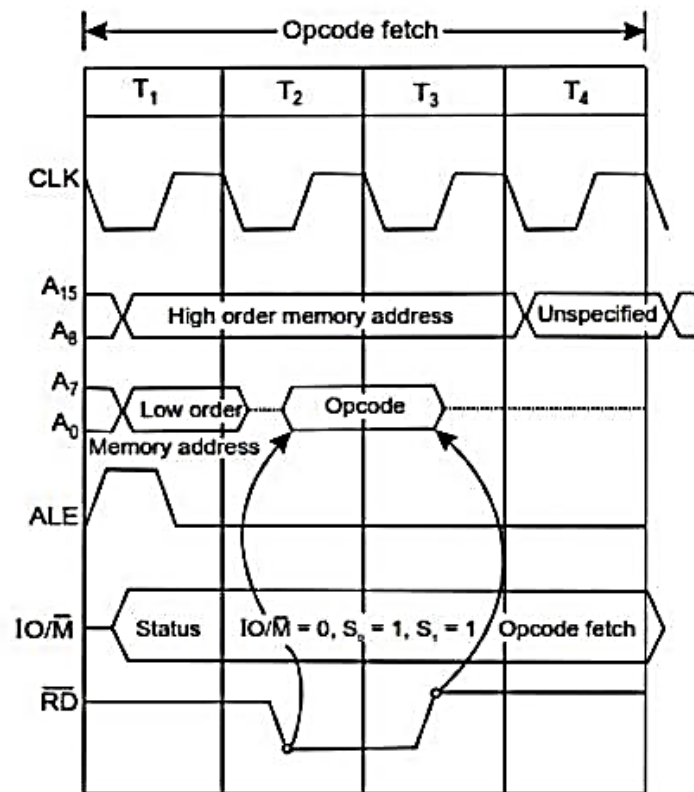


Fig 1.8 Opcode fetch machine cycle

b) Memory Read Machine cycle

- The memory read machine cycle is executed by processor **to read a data byte from memory**.
- The processor **takes 3T states** to execute this cycle.
- The steps in Memory Read machine cycle are

S. No	T state	Operation
1	T ₁	The microprocessor places the higher order 8-bits of the memory address on A ₁₅ – A ₈ address bus and the lower order 8-bits of the memory address on AD ₇ – AD ₀ address / data bus.
2		The microprocessor makes the ALE signal HIGH and at the middle of T ₁ state, ALE signal goes LOW.
3		The status signals are changed as IO/ $\overline{M}$ ' = 0, S ₁ = 1 and S ₀ = 0. These status signals do not change throughout the memory read machine cycle.
4	T ₂	The microprocessor makes the RD' line LOW to enable memory read and increments the Program Counter.
5		The contents on D ₇ – D ₀ (i.e. the data) are placed on the address / data bus.
6	T ₃	The data loaded on the address / data bus is moved to the microprocessor.
7		The microprocessor makes the RD' line HIGH to disable the memory read operation.

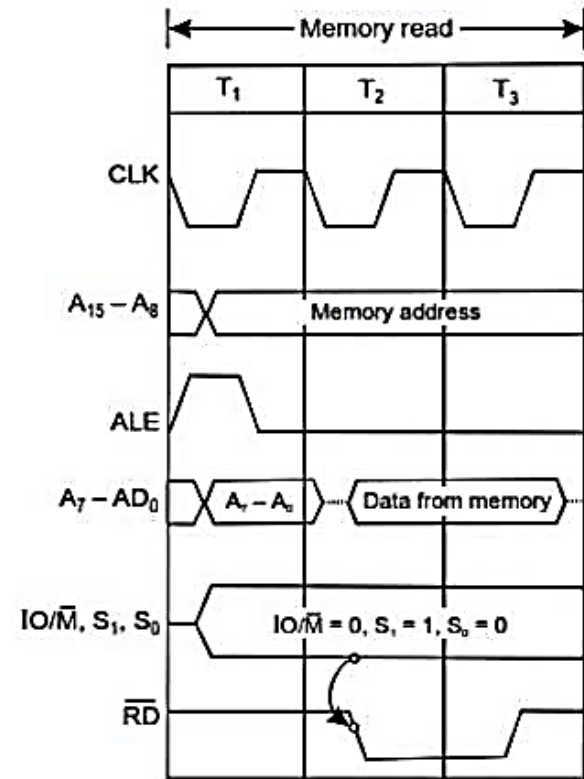


Fig 1.9 Memory Read Machine Cycle

c) Memory Write Machine cycle

- The memory write machine cycle is executed by the processor **to write a data byte** in a memory location.
- The processor takes, **3T states** to execute this machine cycle.

S. No	T state	Operation
1	T ₁	The microprocessor places the higher order 8-bits of the memory address on A ₁₅ – A ₈ address bus and the lower order 8-bits of the memory address on AD ₇ – AD ₀ address / data bus.
2		The microprocessor makes the ALE signal HIGH and at the middle of T ₁ state, ALE signal goes LOW.
3		The status signals are changed as $\overline{IO/\overline{M}} = 0$, $S_1 = 0$ and $S_0 = 1$. These status signals do not change throughout the memory write machine cycle.
4	T ₂	The microprocessor makes the $\overline{WR'}$ line LOW to enable memory write.
5		The contents of the specified register are placed on the address / data bus.
6	T ₃	The data placed on the address / data bus is transferred to the specified memory location.
7		The microprocessor makes the $\overline{WR'}$ line HIGH to disable the memory write operation.

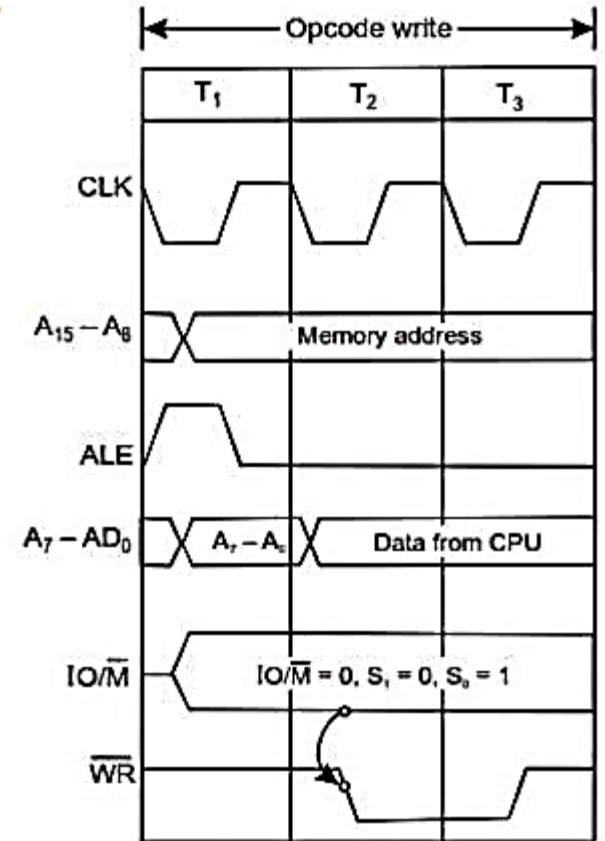


Fig 1.10 Memory Write Machine Cycle

d) I/O Read Machine cycle

- The I/O Read cycle is executed by the processor to read a data byte from I/O port or from the peripheral, which is I/O, mapped in the system.
- The processor takes 3T states to execute this machine cycle.

S. No	T state	Operation
1	T ₁	The microprocessor places the address of the I/O port specified in the instruction on A ₁₅ – A ₈ address bus and also on AD ₇ – AD ₀ address / data bus.
2		The microprocessor makes the ALE signal HIGH and at the middle of T ₁ state, ALE signal goes LOW.
3		The status signals are changed as IO/ $\overline{M}$ ' = 0, S ₁ = 1 and S ₀ = 0. These status signals do not change throughout the I/O read machine cycle.
4	T ₂	The microprocessor makes the $\overline{RD}$ ' line LOW to enable I/O read.
5		The contents on D ₇ – D ₀ (i.e. the data) are placed on the address / data bus.
6	T ₃	The data loaded on the address / data bus is moved to the microprocessor i.e., to the accumulator.
7		The microprocessor makes the $\overline{RD}$ ' line HIGH to disable the I/O read operation.

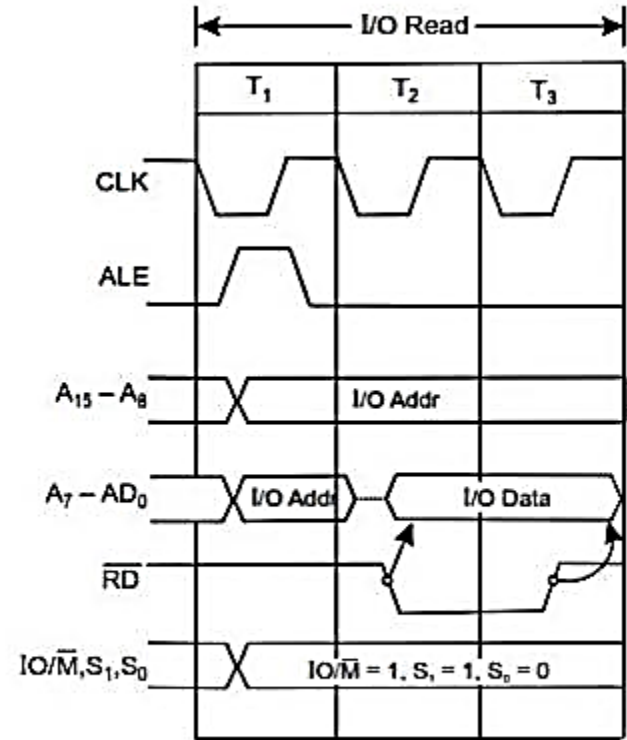
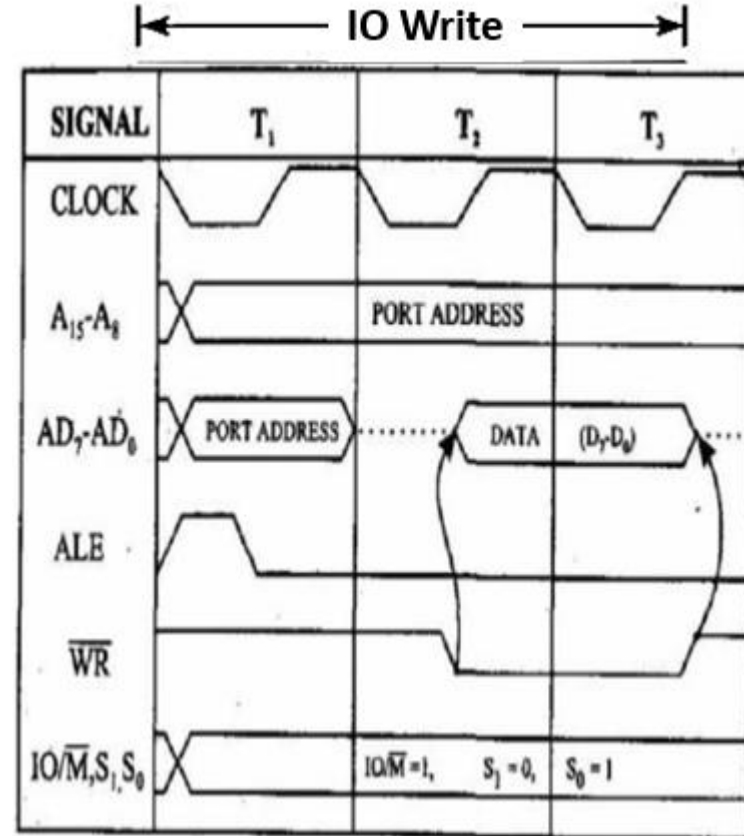


Fig 1.11 I/O Read Cycle

e) I/O Write Machine cycle

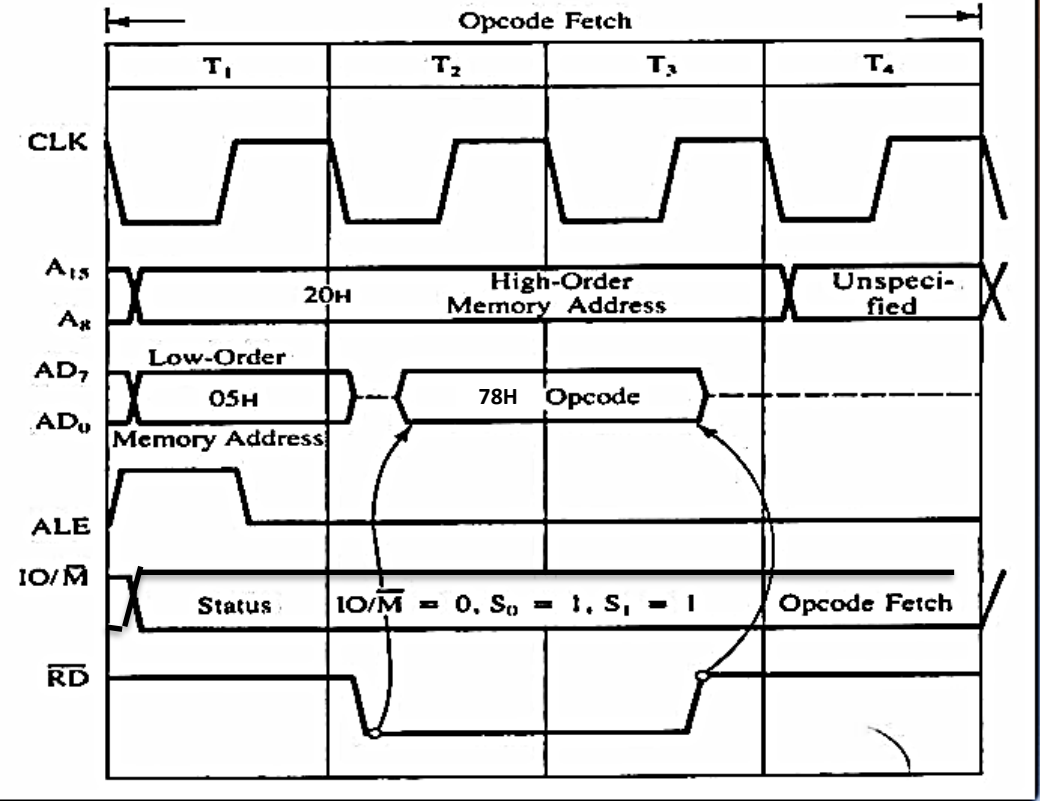
- The I/O Write cycle is executed by the processor **to write a data byte to I/O port** or to the peripheral, which is I/O, mapped in the system.
- The processor **takes 3T states** to execute this machine cycle.

S. No	T state	Operation
1	T ₁	The microprocessor places the address of the I/O port specified in the instruction on A ₁₅ – A ₈ address bus and also on AD ₇ – A ₀ address / data bus.
2		The microprocessor makes the ALE signal HIGH and at the middle of T ₁ state, ALE signal goes LOW.
3		The status signals are changed as IO/M' = 0, S ₁ = 0 and S ₀ = 1. These status signals do not change throughout the I/O write machine cycle.
4	T ₂	The microprocessor makes the $\overline{WR}$ line LOW to enable I/O write.
5		The contents of the Accumulator are placed on the address / data bus.
6	T ₃	The data placed on the address / data bus is transferred to the specified I/O port.
7		The microprocessor makes the $\overline{WR}$ line HIGH to disable the I/O write operation



1. Timing Diagram for: MOV A, B

1. Assume Address: 2005 H, Opcode: 78H
2. Here 20 is higher order address and 05H is Lower Address.



Reference:

<https://www.youtube.com/watch?v=KQ18sRHBpTM>

2. Timing Diagram for: MVI A,43 H

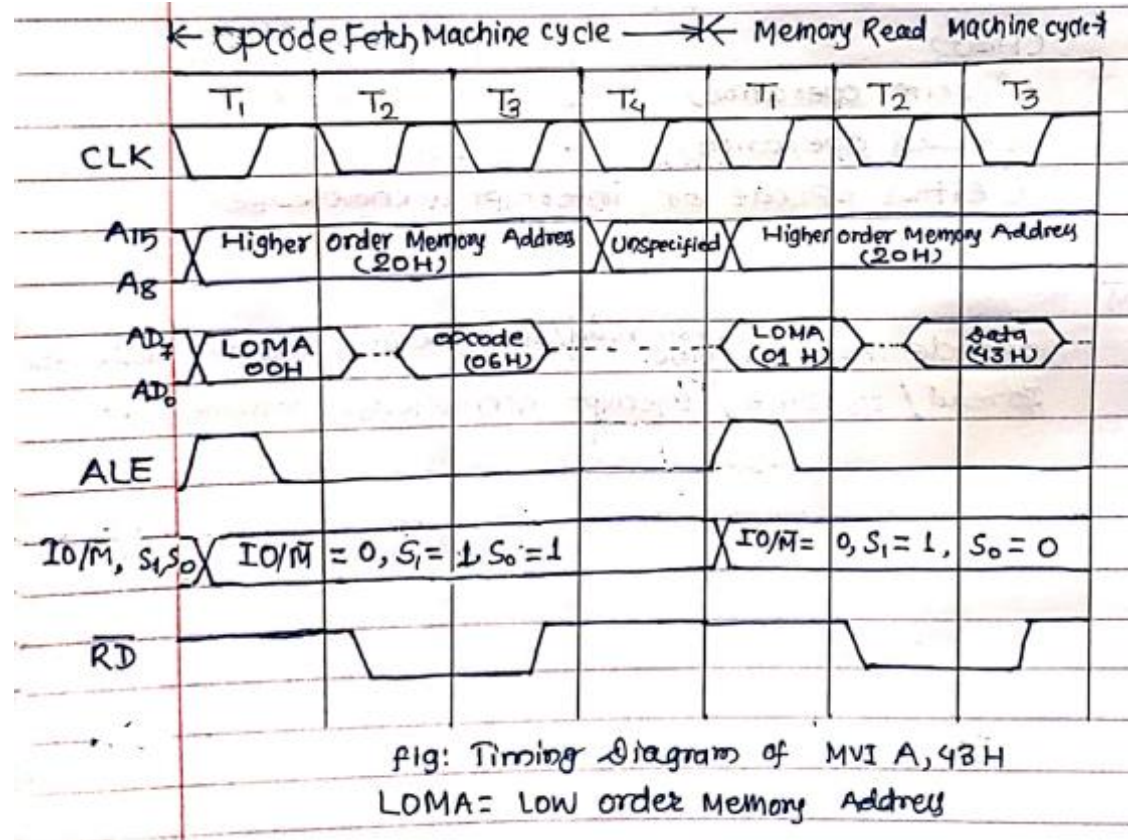
1. Assume

Address	Mnemonics	Hexcode
2000H	MVI A, 443H	06H
2001H		43H

2. Here 20 is higher order address and 00H is Lower Address.

3. Involves:

- Fetching the opcode 06H from the memory 2000H (Opcode Machine Cycle)
- Read (Move) the data 43H from memory 2001H (i.e. Memory Read Machine Cycle)



3. Timing Diagram for: OUT 20H

1. Assume

Address	Mnemonics	Hexcode
2050H	OUT	D3H
2051H		20H

2. Here 20 is higher order address and 50H is Lower Address.

3. Involves:

- **Opcode Machine Cycle :** Fetching the opcode D3H from the memory 2050H
- **Memory Read Machine Cycle:** Read the port address 20H from the memory 2051H
- **IO Write Machine cycle:** Transfer the content of Accumulator to the output port 20H

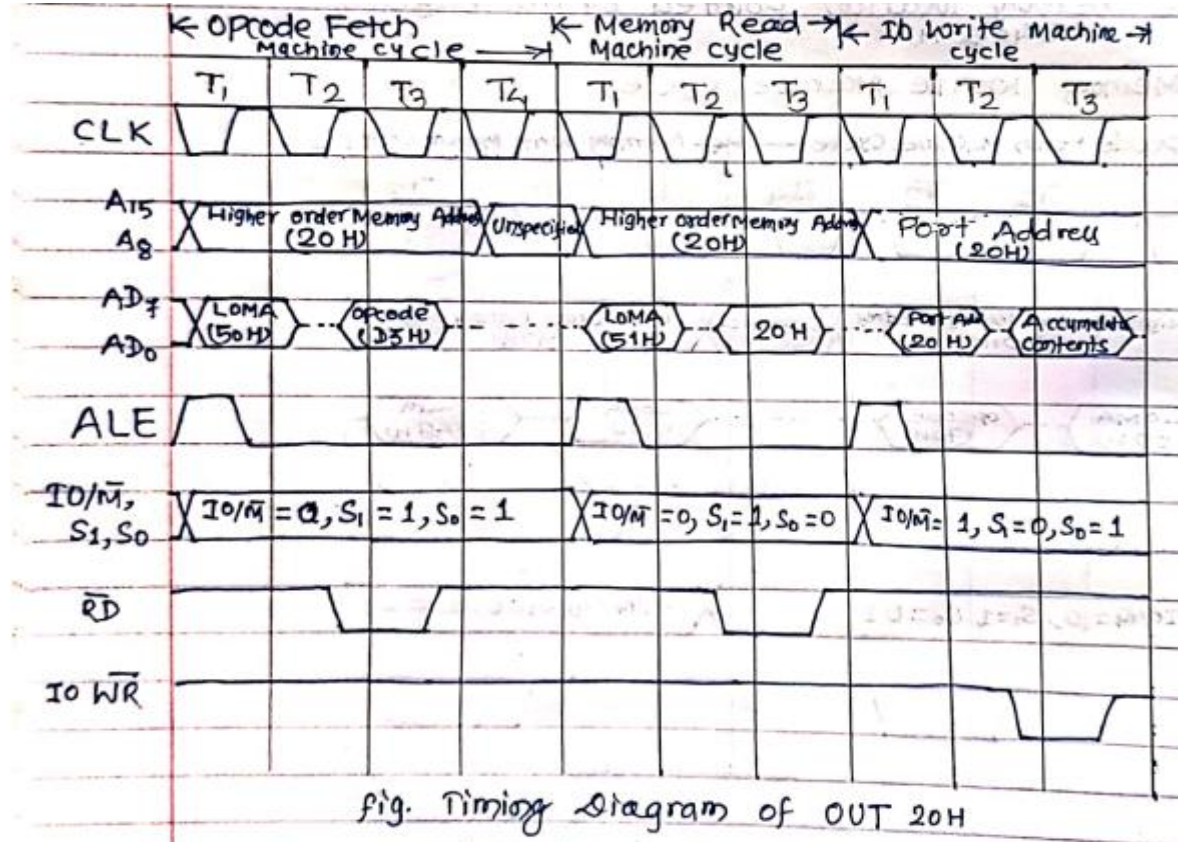


fig. Timing Diagram of OUT 20H

4. Timing Diagram for: STA 2050H

1. Assume

Address	Mnemonics	Hexcode
1080H	STA	32H
1081H		50H
1082H		20H

2. The instruction copies the accumulator into the memory location 2050H. It is a 3-byte instruction and the MP needs four machine cycles (13 T-state) to complete its execution.

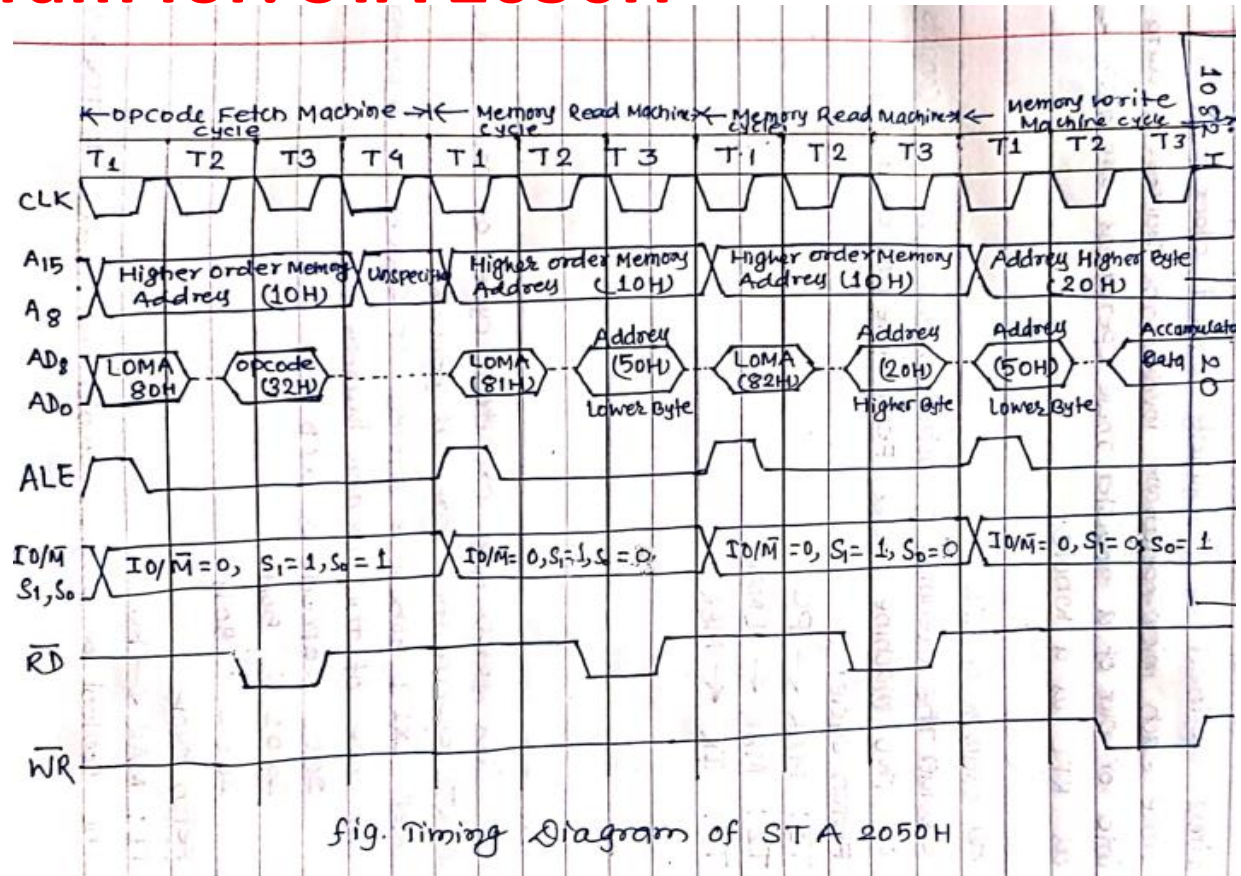
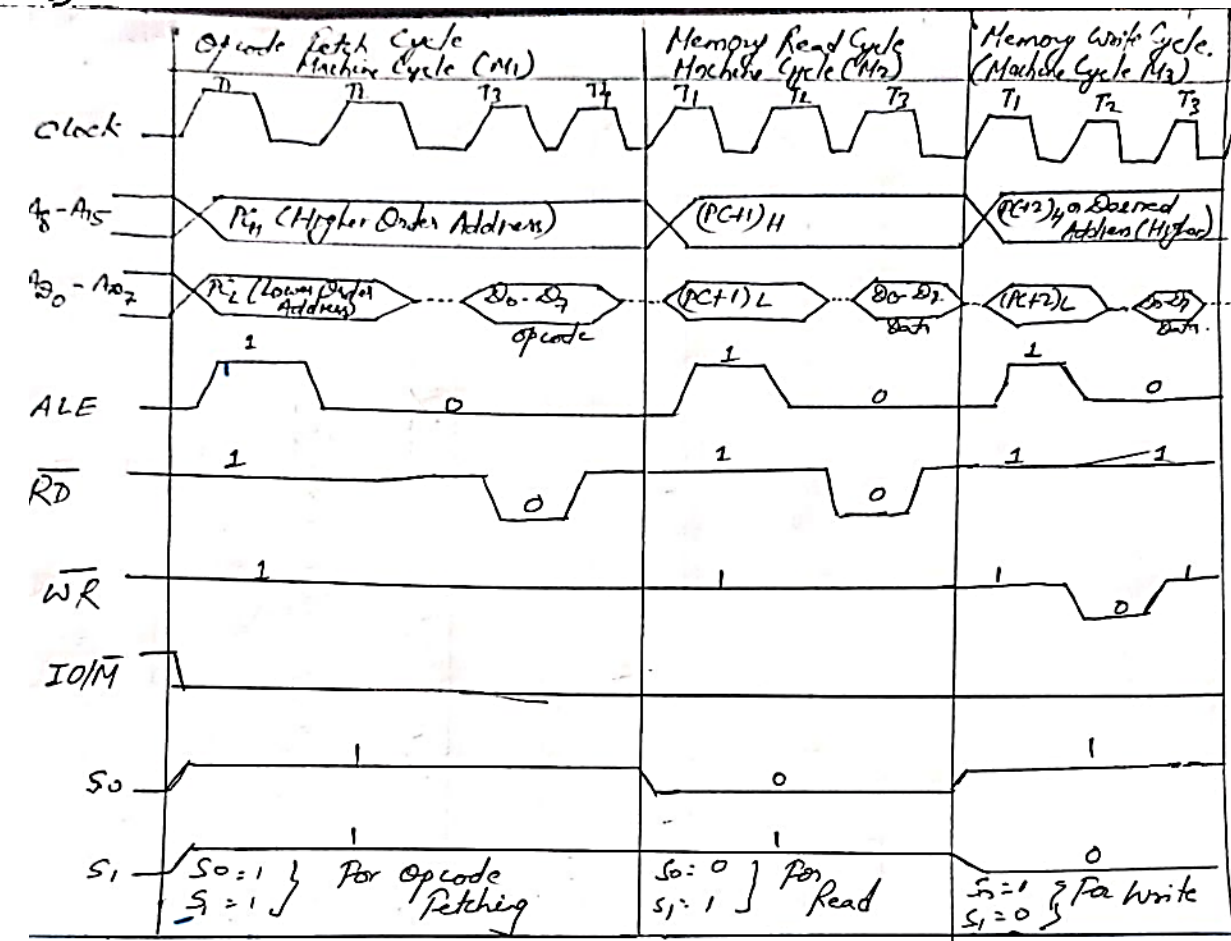


fig. Timing Diagram of STA 2050H

8085 Microprocessor (Timing Diagram).

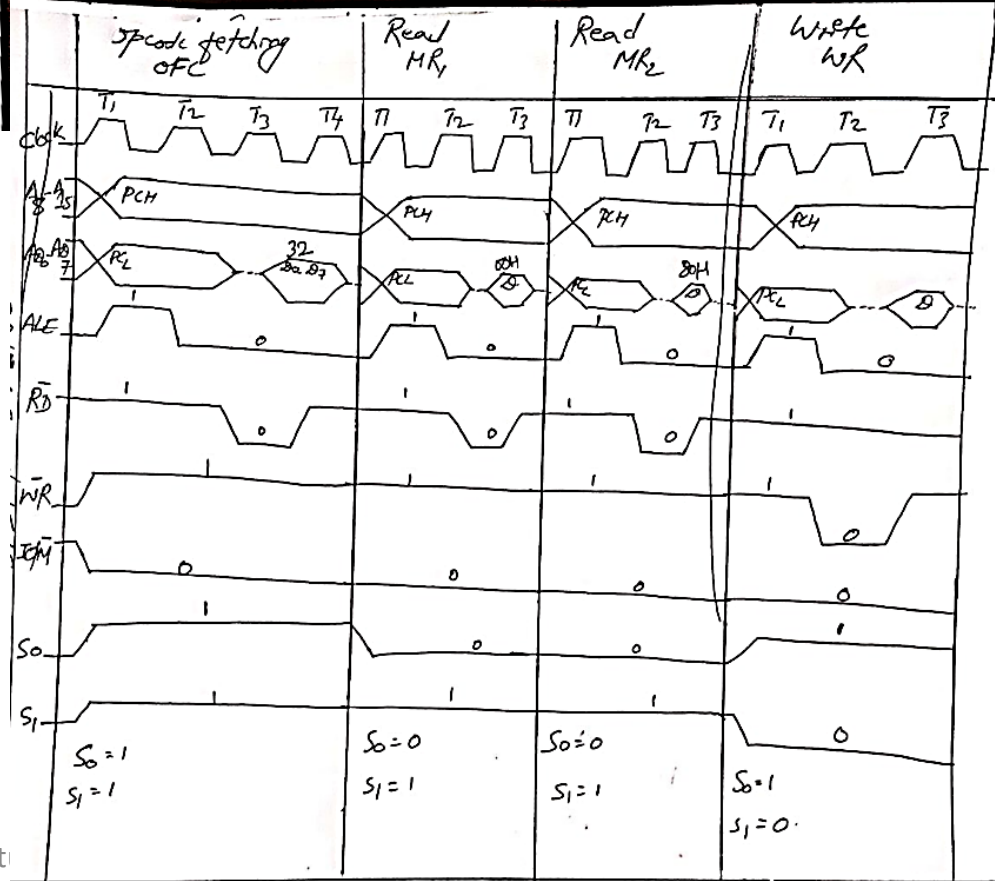
- Read, Write & Both Operation.
- 1) Read Operation.
 - 2) Write Operation.
 - 3) Read (Write).



Eg. Timing Diagram of STA 8000H
 Opcode $\rightarrow$ 32H } 1 byte For opcode fetching 4T is required.
 Operand $\rightarrow$ 00H } 2 byte.
 80H } To read the operand: 3+3=6T

To write data we need 3T state.

Total t-state = $3+3+3+4=13$ T-state.



The flow of an Instruction in 8085 architecture

1. Execution starts with PC. It starts the program execution with next address field. It fetches the instruction from the memory location pointed by PC.
2. For address fetching from memory, multiplexed address/data bus acts as an address bus and after fetching instruction, this address bus will now act as a data bus and extract data from the specified memory location, and send this data on an 8-bit internal bus. For multiplexed address/data bus, address Latch Enable (ALE) pin is used. This allows the input address to be entered to a register. If ALE=1, multiplexed bus is address bus, otherwise it acts as Data bus.
3. After data fetching, it will go into the IR. It will store data fetched from memory and data is now ready for decoding. (Instruction decoder is used)
4. After that, Timing and control comes into the account. It sends control signals to all over the microprocessor to tell it whether the given instruction is for READ/WRITE, and whether it is for Memory or IO devices activity.
5. According to the timing and control signal pins, logical and arithmetic operations are performed (ALU).
6. With the help of Serial I/O data pin (SID and SOD), we can send or receive input/output to external devices. In this way, execution cycle is carried out.
7. While execution is going on if there is any interrupt detected, then it will stop execution of the current process and invoke ISR (Interrupt service routine) function, which will stop the current execution and do the execution of current occurred interrupt.

5. Basic Assembly language programs of 8085



1. Write an ALP for 8085 MP to store an immediate 8-bit data in memory location 2051H using direct addressing.

MVI A,49H ; "Store 49H in the accumulator"

STA 2501H ; "Copy accumulator contents at address 2501H"

HLT ; "Stop"



2. Write an ALP for 8085 MP to store 8-bit data in memory 2501H using indirect addressing.

```
LXI H,2501H    ; "Load H-L pair with 2501H"  
MVI M,49H      ; "Store 49H in memory location pointed by H-L register pair (2501H)"  
HLT            ; "Stop"
```



3. Write an ALP for adding two 8-bit numbers 02H and 03H using immediate addressing . The result is to be stored in 1050 H.

```
MVI A, 02H    ; "Get the first number and move to accumulator"  
MVI B, 03H    ; "Get the second number and move to register B"  
ADD B         ; "Add the second number with the content of accumulator"  
STA 1050H     ; "Store result at 1050H"  
HLT           ; "Stop"
```



4. Write an ALP for adding two 8-bit numbers stored in 2501H and 2502H. The result is to be stored in 2503H.

```
LXI H, 2501H ; "Get address of first number in H-L pair. Now H-L points to 2501H"  
MOV A, M     ; "Get first operand in accumulator"  
INX H        ; "Increment content of H-L pair. Now, H-L points 2502H"  
ADD M        ; "Add first and second operand"  
INX H        ; "H-L points 2503H"  
MOV M, A     ; "Store result at 2503H"  
HLT          ; "Stop"
```

5. Write an ALP for subtracting two 8-bit numbers stored in 2501H and 2502H. The result is to be stored in 2503H.



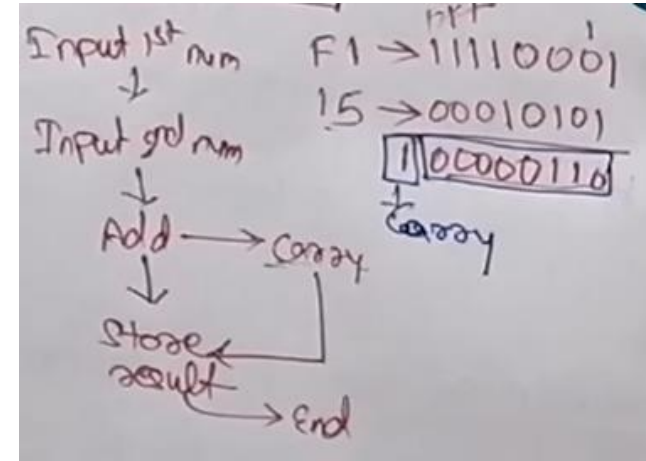
```
LXI H, 2501H ; "Get address of first number in H-L pair. Now H-L points to 2501H"
MOV A, M      ; "Get first operand in accumulator"
INX H         ; "Increment content of H-L pair. Now, H-L points 2502H"
SUB M         ; "Subtract first to second operand"
INX H         ; "H-L points 2503H"
MOV M, A      ; "Store result at 2503H"
HLT           ; "Stop"
```


6. Write an assembly language program to add two 8-bit numbers (F1 and 15), stored at C050H and C051H, with carry.

```

LXI H, C050H ; "Get address of first number in H-L pair. Now H-L points to C050H"
MOV A, M     ; "Get first operand in accumulator from C050H"
INX H       ; "Increment content of H-L pair. Now, H-L points C051H"
MVI B, 00H   ; "Initialize Register B with 00H for carry store"
ADD M       ; "Add first and second operand, result stored in Acc"
JNC label    ; "Jump to label if no carry"
INR B       ; "Increment content of B by 1"
label: STA C053H ; "Store the content of Acc to C053H"
MOV A, B    ; "Move carry at Acc"
STA C052H   ; "Store the content of Acc to C052H, ie the carry part"
HLT        ; "Stop"
           ; "Carry part store in C052H and sum is store in C053H"

```



Reference:

<https://www.youtube.com/watch?v=GLfEQGY70Ek&list=PLndX8heiWwErijxt7FpnRilO2KV4UhFmN>

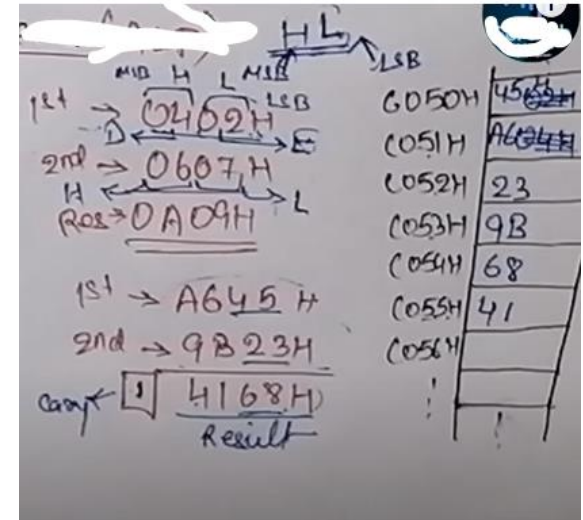
7. Write an assembly language program to add two 16-bit numbers A645 and 9B23, stored from C050H onwards, with carry.

```

LHLD C050H ; "Get address of first number from memory and load in in H-L register"
XCHG      ; Exchange H and L with D and E
LHLD C052H ; "Get address of second number from memory and load in in H-L register"
MVI B, 00H ; "Initialize Register B with 00H for carry store"
DAD D     ; Add both the register pairs and store in HL, ie. HL+DE
JNC label ; "Jump to label if no carry"
INR B     ; "Increment content of B by 1"
label: SHLD C054H ; "Store the content of H-L register to C054H"
MOV A, B  ; "Move carry at Acc"
STA C056H ; "Store the content of Acc to C056H, ie the carry part"
HLT      ; "Stop"

```

; "Carry part store in C056H and sum is store in C054H-C055H"



Reference: https://www.youtube.com/watch?v=_4Q-upolv8c&list=PLndX8heiWwErijxt7FpnRilO2KV4UhFmN&index=2



8. Write a program in Assembly language to interchange(Swap) the contents of two memory location 2100H and 2101H.

```
LDA 2100H    ; "Load the content of 2100H into Accumulator"  
MOV B,A      ; "Move content of Acc to register B"  
LDA 2101H    ; "Load content of 2101H to Accumulator"  
STA 2100H    ; "Store content of Acc. to 2100H"  
MOV A,B      ; "Move content of register B to Acc."  
STA 2101H    ; "Store content of Acc. To 2101H"  
HLT          ; "End"
```



8. Write a program in Assembly language to multiply two 8 bit numbers stored at C050H and C051H.

	LXI H,C050H	; Get address of first number in H-L pair. Now H-L points to C050H"	
	MOV B,M	; Get first operand in register B from C050H"	
	MVI A,00H	;Clear Accumulator	
	MVI D, 00H	;Initialize a register D with 00H for carry storage	
	INX H	; point to C051H	
	MOV C,M	;Get the second operand	
loop1:	ADD B	;Add contents in register B with Accumulator	5x3*15
	JNC loop2	; Jump to loop2 if no carry	
	INR D	;increase D	5+5+5=15
loop2:	DCR C	;Decrement counter C	
	JNZ loop1	;Jump to loop1	
	STA C052H	;Store the content to C052H	
	MOV A,D	;Move D to A.	
	STA C053H	;Store content of Acc. To C053.	
	HLT		

Assignment:

1. Write a program, which load register B with 37H and copy the number in accumulator and send the number to the memory location 2000H.
2. Write a program, which load 37H and 45H. Find the sum of two number and send the result in 2050H.
3. Write assembly language program using the instruction of 8085. Load the contents of memory location 2100H and 2101H in B register and C-Register respectively. The content of memory location 2100H and 2101H are 16H and 19 H respectively.
4. Write an assembly language program to multiply two decimal numbers 23 and 9 and store the answer in some memory location.

A - C		D - L		L - M		M - P		P - S		S - X	
HEX	Mnemonic	HEX	Mnemonic	HEX	Mnemonic	HEX	Mnemonic	HEX	Mnemonic	HEX	Mnemonic
CE	ACI data	27	DAA	11	LXI D	63	MOV H.E	E5	PUSH HL	95	SUB L
8F	ADC A	09	DAD B	21	LXI H	64	MOV H.H	C5	PUSH PC	96	SUB M
88	ADC B	19	DAD D	31	LXI SP	65	MOV H.L	F5	PUSH PSW	D6	SUI data
89	ADC C	29	DAD H	7F	MOV A.A	66	MOV H.M	17	RAL	AF	XRA A
8A	ADC D	39	DAD SP	78	MOV A.B	6F	MOV L.A	1F	RAR	A8	XRA B
8B	ADC E	3D	DCR A	79	MOV A.C	68	MOV L.B	D8	RC	A9	XRA C
8C	ADC H	05	DCR B	7A	MOV A.D	69	MOV L.C	C9	RET	AA	XRA D
8D	ADC L	0D	DCR C	7B	MOV A.E	6A	MOV L.D	20	RIM	AB	XRA E
8E	ADC M	15	DCR D	7C	MOV A.H	6B	MOV L.E	07	RLC	AC	XRA H
87	ADD A	1D	DCR E	7D	MOV A.L	6C	MOV L.H	F8	RM	AD	XRA L
80	ADD B	25	DCR H	7E	MOV A.M	6D	MOV L.L	D0	RNC	AE	XRA M
81	ADD C	2D	DCR L	47	MOV B.A	6E	MOV L.M	C0	RNZ	EE	XRI data
82	ADD D	35	DCR M	40	MOV B.B	77	MOV M.A	F0	RP	E3	XTHL
83	ADD E	0B	DCX B	41	MOV B.C	70	MOV M.B	E8	RPE	EB	XCHG
84	ADD H	1B	DCX D	42	MOV B.D	71	MOV M.C	E0	RPO		
85	ADD L	2B	DCX H	43	MOV B.E	72	MOV M.D	0F	RRC		
86	ADD M	3B	DCX SP	44	MOV B.H	73	MOV M.E	C7	RST 0		
C6	ADI data	F3	DI	45	MOV B.L	74	MOV M.H	CF	RST 1		
A7	ANA A	FB	EI	46	MOV B.M	75	MOV M.L	D7	RST 2		
A0	ANA B	76	HLT	4F	MOV C.A	3E	MVI A.data	DF	RST 3		
A1	ANA C	DB	IN port	48	MOV C.B	06	MVI B.data	E7	RST 4		
A2	ANA D	3C	INR A	49	MOV C.C	0E	MVI C.data	EF	RST 5		
A3	ANA E	04	INR B	4A	MOV C.D	16	MVI D.data	F7	RST 6		
A4	ANA H	0C	INR C	4B	MOV C.E	1E	MVI E.data	FF	RST 7		
A5	ANA L	14	INR D	4C	MOV C.H	26	MVI H.data	C8	RZ		
A6	ANA M	1C	INR E	4D	MOV C.L	2E	MVI L.data	9F	SBB A		
E6	ANI data	24	INR H	4E	MOV C.M	36	MVI M.data	98	SBB B		
CD	CALL addr	2C	INR L	57	MOV D.A	00	NOP	99	SBB C		
DC	CC addr	34	INR M	50	MOV D.B	ED	NOP	9A	SBB D		
FC	CM addr	03	INX B	51	MOV D.C	DD	NOP	9B	SBB E		
2F	CMA	13	INX D	52	MOV D.D	B7	ORA A	9C	SBB H		
3F	CMC	23	INX H	53	MOV D.E	B0	ORA B	9D	SBB L		
BF	CMP A	33	INX SP	54	MOV D.H	B1	ORA C	9E	SBB M		
B8	CMP B	DA	JC addr	55	MOV D.L	B2	ORA D	DE	SBI data		
B9	CMP C	FA	JM addr	56	MOV D.M	B3	ORA E	22	SHLD addr		
BA	CMP D	C3	JMP addr	5F	MOV E.A	B4	ORA H	30	SIM		
BB	CMP E	D2	JNC addr	58	MOV E.B	B5	ORA L	F9	SPHL		
BC	CMP H	C2	JNZ addr	59	MOV E.C	B6	ORA M	32	STA addr		
BD	CMP L	F2	JP addr	5A	MOV E.D	F6	ORI data	02	STAX B		
BE	CMP M	EA	JPE addr	5B	MOV E.E	D3	OUT port	12	STAX D		
D4	CNC addr	E2	JPO addr	5C	MOV E.H	E9	PCHL	37	STC		
C4	CNZ addr	CA	JZ addr	5D	MOV E.L	C1	POP BC	97	SUB A		
F4	CP addr	3A	LDA addr	5E	MOV E.M	D1	POP DE	90	SUB B		
EC	CPE addr	0A	LDAX B	67	MOV H.A	E1	POP HL	91	SUB C		
FE	CPI data	1A	LDAX D	60	MOV H.B	F1	POP PSW	92	SUB D		
E4	CPO addr	2A	LHLD addr	61	MOV H.C	C5	PUSH BC	93	SUB E		
CC	CZ addr	01	LXI B	62	MOV H.D	D5	PUSH DE	94	SUB H		

Register Transfer Language (RTL)

Register Transfer Language (RTL)

- The symbolic notation used to describe the micro-operation transfers amongst registers is called **Register transfer language**.
- Register Transfer Language, RTL, (sometimes called register transfer notation) is a powerful high level method of describing the architecture of a circuit.
- Information transfer from one register to another is designated in symbolic form by means of a replacement operator.
- For example: $R2 \leftarrow R1$
 - The statement denotes a transfer of the content of register R1 into register R2. It designates a replacement of the content of R2 by the content of R1.
 - By definition, the content of the source register R1 does not change after the transfer.

Register Transfer

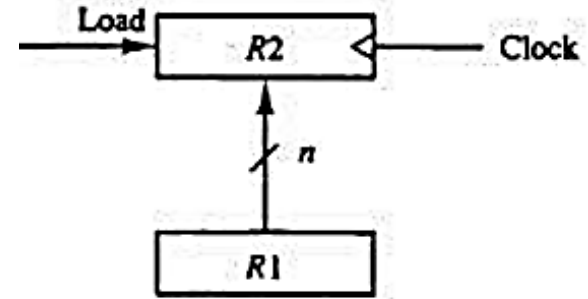
- Information transferred from one register to another is designated in symbolic form by means of replacement operator.

$R2 \leftarrow R1$

- It denotes the transfer of the data from register R1 into R2.
- Normally we want the transfer to occur only in predetermined control condition. This can be shown by following **if-then** statement:

if (P=1) then ($R2 \leftarrow R1$)

- Here, P is a control signal generated in the control section.



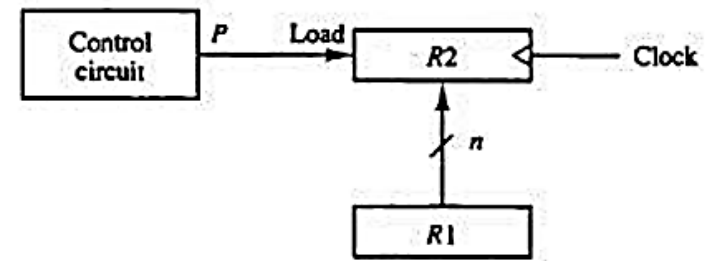
Control Function

- A control function is a Boolean variable that is equal to 1 or 0. The control function is shown as:

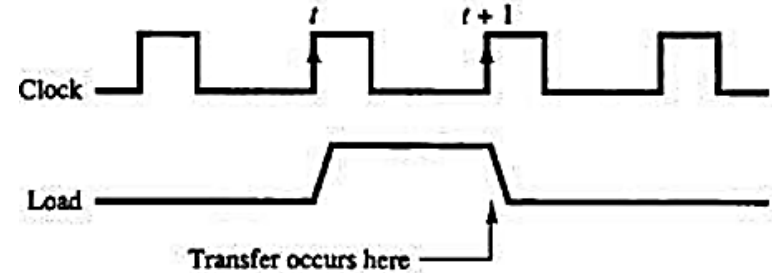
P: $R2 \leftarrow R1$

- The control condition is terminated with a colon.
- It shows that transfer operation can be executed only if $P=1$.
- If two or more operations are to occur simultaneously, they are separated with commas.
- Example:

P: $R2 \leftarrow R1, MAR \leftarrow IR$



(a) Block diagram



(b) Timing diagram

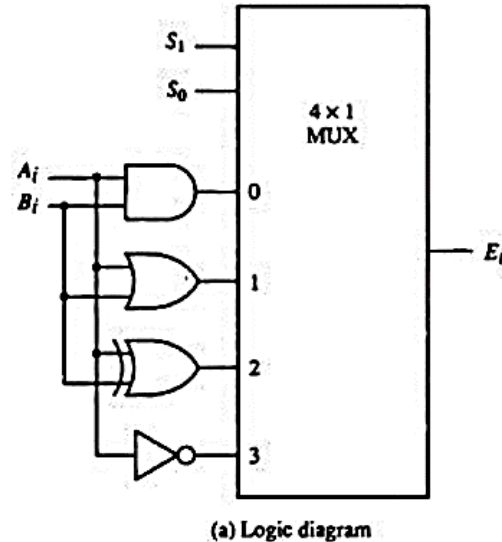
Arithmetic Microoperation

- The operations executed on data stored in registers are called **microoperations**.
- Examples of microoperations are shift, count, clear and load

Symbolic Designation	Description
$R3 \leftarrow R1 + R2$	Contents of R1 plus R2 transferred to R3
$R3 \leftarrow R1 + R2' + 1$	Contents of R1 minus R2 transferred to R3
$R1 \leftarrow R1 + 1$	Increment the contents of R1 by one
$R1 \leftarrow R1 - 1$	Decrement the contents of R1 by one
$R3 \leftarrow R1 + R2 + 1$	Add with carry
$R3 \leftarrow R1 + R2'$	Subtract with borrow
$R1 \leftarrow R1' + 1$	2's complement the contents of R1 (negate)

Logic Microoperation

Microoperation	Name
$F \leftarrow R1 \wedge R2$	AND
$F \leftarrow R1 \vee R2$	OR
$F \leftarrow R1 \oplus R2$	XOR
$F \leftarrow R1'$	Complement (NOT)



S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

(b) Function table

Figure: Hardware Implementation

Shift Microoperation

Microoperation	Name
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr } R$	Shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic left shift register R
$R \leftarrow \text{ashr } R$	Arithmetic right shift register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R

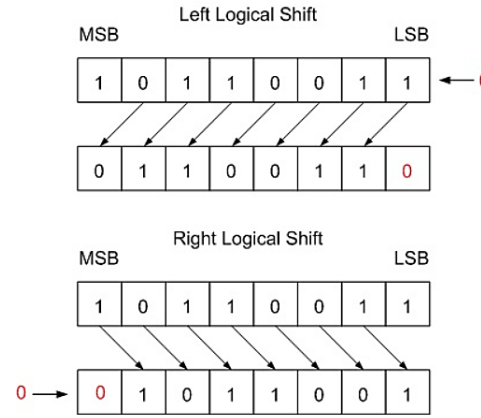


Fig: Logical shift by 1 bit

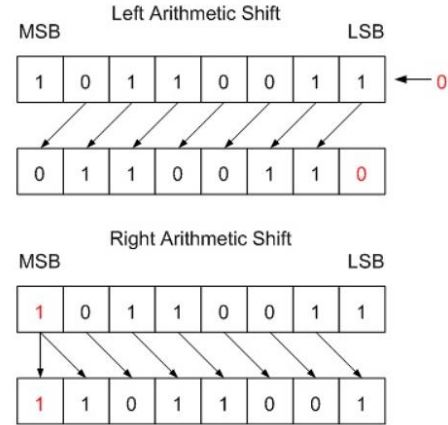


Fig: Arithmetic shift by 1 bit

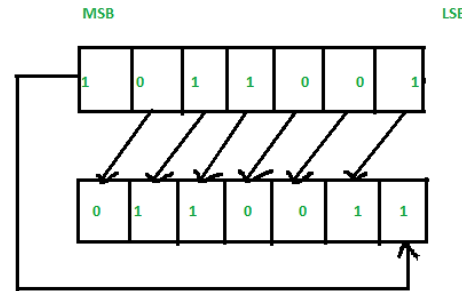


Fig: Circular left shift by 1 bit

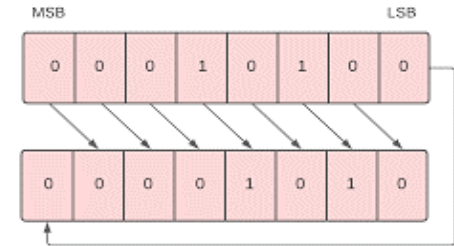


Fig: Circular right shift by 1 bit

Specifying with RTL for LXI D, 9050H

- This instruction loads 16-bit data 9050H in register pair DE such that Low-order byte 50H is loaded into the register D.
- Let LXI D, 9050H is stored at 2000H. So the corresponding hex-code of the memory locations are:
 - 2000: opcode of LXI D
 - 2001: 50H
 - 2002: 90H
- The fetch and execution cycle of this instruction in RTL specification are as follows:

2000H:	Opcode of LXI D
2001H:	50H
2002H:	90H
2003H:	
2004H:	
2005H:	

Memory

a) Fetch Cycle:

- **T1: $MAR \leftarrow PC$**
 - The **content of PC is moved to the MAR.**
 - The PC contains the address of next instruction to be executed.
 - MAR is 16-bit register that holds the address of memory temporarily during read/write operation.
 - So, now MAR has 2000H.
- **T2: $MBR \leftarrow [MAR]$**
 - The **content of memory pointer by MAR is moved to MBR.**
 - MBR is memory buffer register that holds data temporarily before writing in memory or after reading from the memory.
 - So, MBR will now have opcode of LXI D
- **T3: $IR \leftarrow MBR, PC \leftarrow PC+1$**
 - The **content of MBR** (ie. opcode of LXI D) **is now sent to IR.**
 - The content of IR is decoded and **PC increment by 1.**
- **T4: unspecified**

b) Execute cycle:

Machine cycle M1

- **T1: $MAR \leftarrow PC$**
 - The **new content of PC** (i.e 2001) is sent to **MAR**
- **T2: $MBR \leftarrow [MAR]$**
 - The content of location 2001H (**ie. 50H**) **is sent to MBR**
- **T3: $E \leftarrow MBR, PC \leftarrow PC+1$**
 - This lower order byte 50H is sent to register E and PC is incremented by 1.

Machine cycle M2

- **T1: $MAR \leftarrow PC$**
 - The new content of PC, ie. 2002H is sent to MAR
- **T2: $MBR \leftarrow [MAR]$**
 - The content of location 2002H, ie. 90H is sent to MBR
- **T3: $D \leftarrow MBR$**
 - The higher order byte 90H is sent to register D.

STA 2000H using RTL specification

- This instruction copies the contents of the accumulator to the memory location 2000H. It is 3-byte instruction and requires 4 machine cycles (13T states) for its execution.
- The RTL for this instruction are shown below:

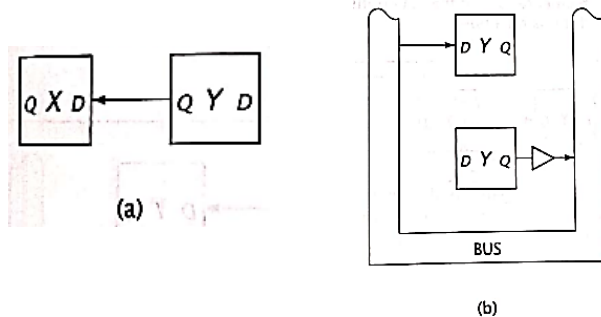
Fetch cycle	
T1:	$MAR \leftarrow PC$
T2:	$MBR \leftarrow [MAR]$
T3:	$IR \leftarrow MBR, PC \leftarrow PC + 1$
T4:	Unspecified

Execute cycle	
M1: T1:	$MAR \leftarrow PC$
T2:	$MBR \leftarrow [MAR]$
T3:	$Z \leftarrow MBR, PC \leftarrow PC + 1$
M2: T1:	$MAR \leftarrow PC$
T2:	$MBR \leftarrow [MAR]$
T3:	$W \leftarrow MBR, PC \leftarrow PC + 1$
M3: T1:	$MAR \leftarrow WZ$
T2:	$MBR \leftarrow A$
T3:	$[MAR] \leftarrow MBR, PC \leftarrow PC + 1$

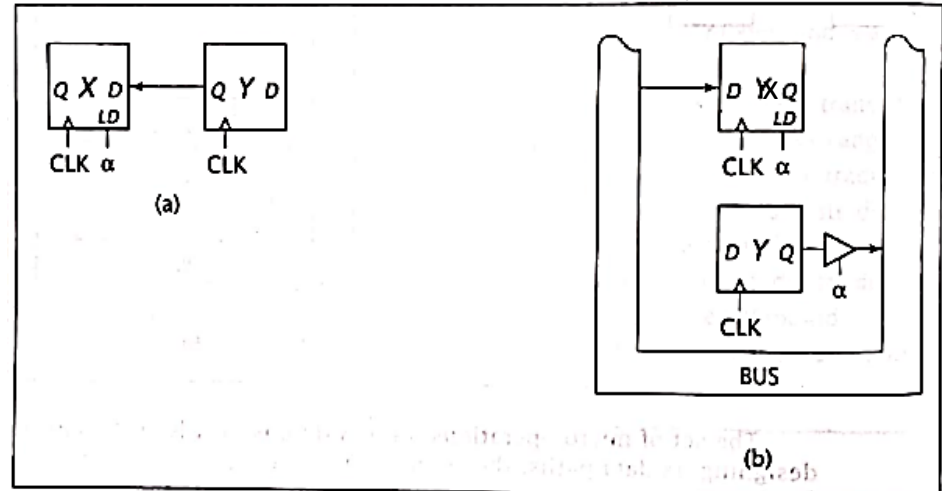
Some RTL implementation

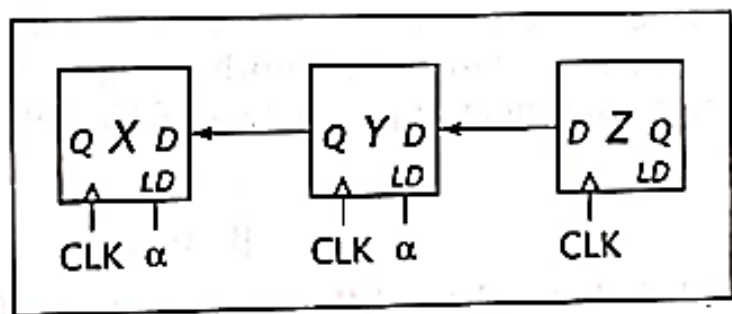
- RTL can be used to specify the behavior of any sequential digital system, simplest to complex.
- RTL specify the function of digital components and simple systems.

Implementations of the data transfer $\alpha: X \leftarrow Y$ with control signals: (a) with direct path, and (b) using a bus

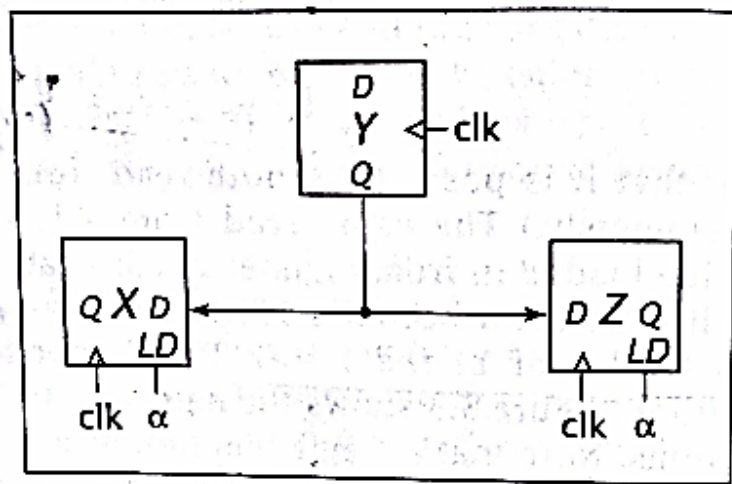


Implementations of the micro-operation $X \leftarrow Y$ using (a) a direct connection and, (b) a bus connection





Implementation of the data transfer $\alpha: X \leftarrow Y, Y \leftarrow Z$



Implementation of the data transfer $\alpha: X \leftarrow Y, Z \leftarrow Y$

Representation of D flipflop with RTL

- Consider the D flip-flop as shown in figure (a).
- Its function can be expressed by single RTL statement.

LD: $Q \leftarrow D$

- When **LD input is high**, the **value on the D input is loaded** and made available on the **Q output**. This happens on rising edge of clock pulse.
- In figure b, when $CLR = 1$, the flipflop set to 0.

i.e.

LD: $Q \leftarrow D$

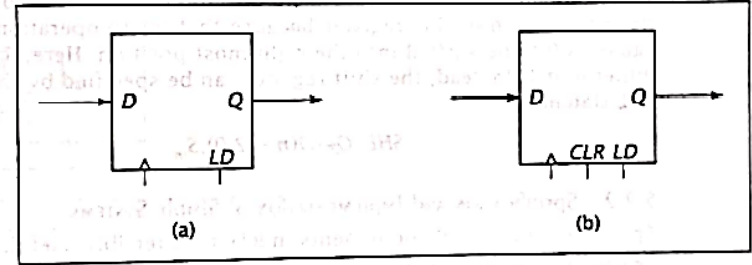
CLR: $Q \leftarrow 0$

- However, When D, LD and CLR are equal to 1, the above system fails.
 - When $LD=1$, the Q is set to 1
 - And when $CLR=1$, Q is clear to 0,
 - Both set Q simultaneously, Which is not possible.
- The solution is to modify the conditions so they are mutually exclusive.

$CLR'LD: Q \leftarrow D$

- Here, Q is set to 1 if $CLR=0$ and $LD=1$.

D flip-flop: (a) without and (b) with clear input



CLK	D	Q_{n+1}
0	x	Q_n
1	0	0
1	1	1

D Flipflop truth table

Another Example: JK

- Consider the JK flip flop without a CLR input.
- Its truth table is:

Truth Table			
J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\overline{Q}_0$ (toggles)

- Its behavior can be specified in RTL as follow:

$J'K: Q \leftarrow 0$

$JK': Q \leftarrow 1$

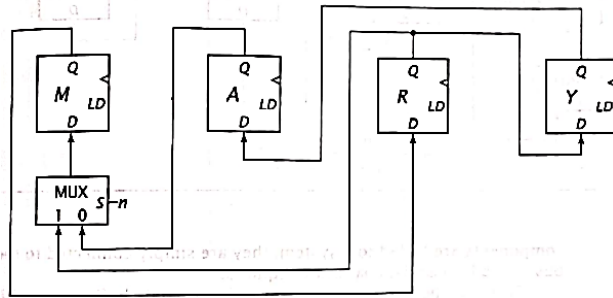
$JK: Q \leftarrow Q'$

- When $J=0$ and $K=0$, no condition is met and the flip-flop retains its previous value. No RTL statement is included for this case because no transfer is needed.

Specification and Implementation of Simple Systems with RTL

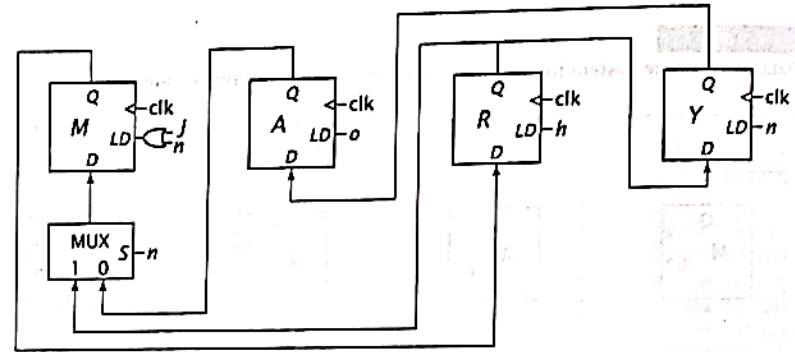
- Consider a system with four 1-bit flipflops. Given RTL code where j, o, h and n are conditions which are mutually exclusive.

$j: M \leftarrow A$
 $o: A \leftarrow Y$
 $h: R \leftarrow M$
 $n: Y \leftarrow R, M \leftarrow R$



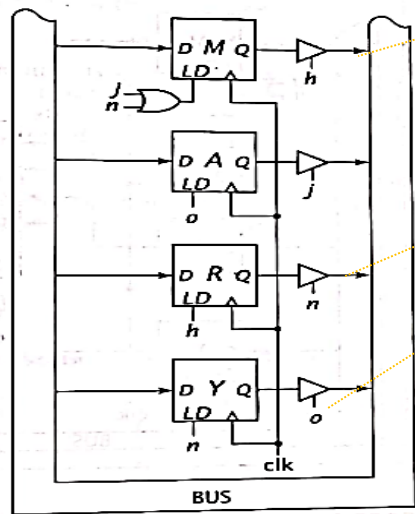
Data paths of the system to implement the RTL code using direct connections

- But it does not complete the transfer.
- No register has actually loaded any data yet.**
- It is necessary to assert LD signal.
- The complete circuit, with LD signal is shown as below.
- The flipflop M is loaded when $j=1$ ($M \leftarrow A$) or $n=1$ ($M \leftarrow R$).
- Logically, ORing these signals together produces LD input for M flipflop.
- Similarly for other conditions.



Complete design of the system to implement the RTL code using direct connections

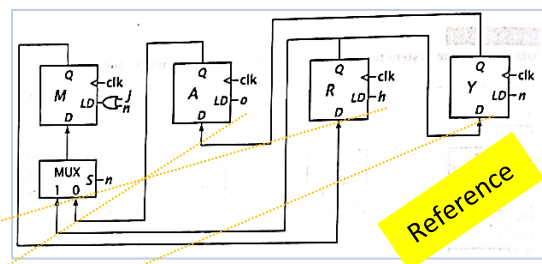
- Buses can be implemented using either tri-state buffers or multiplexers.
- Figure aside shows the complete design of the system to implement RTL code using a bus and tri-state buffers.



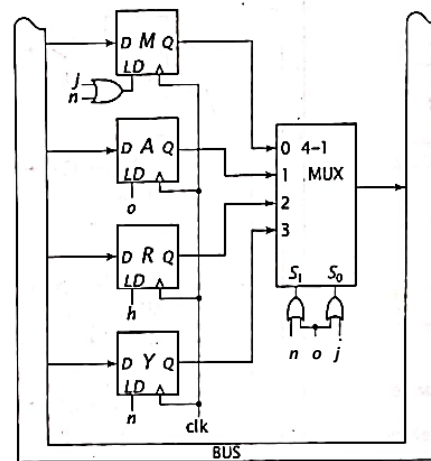
Complete design of the system to implement the RTL code using a bus and tri-state buffers

8/1/2023

j: $M \leftarrow A$
o: $A \leftarrow Y$
h: $R \leftarrow M$
n: $Y \leftarrow R, M \leftarrow R$



- It is also possible to implement a system bus using a multiplexer.
- MUX can be used instead of tri-state buffers.



Complete design of the system to implement the RTL code using a bus and multiplexer

90

Classwork:

1 Write valid RTL statements that realize the following transitions. All registers are 1-bit wide.

- a) IF $\alpha = 1$ THEN copy X to W and copy Z to Y
- b) IF $\alpha = 1$ THEN copy X to W ; otherwise copy Z to Y
- c) IF $\alpha = 0$ THEN copy X to W

2 Show the hardware to implement the RTL statements developed for Problem 1.

3 Write the RTL code for the following transitions. All registers are 1-bit wide. Signals α and β are never equal to 1 simultaneously.

- a) IF $\alpha = 1$ THEN copy Y to X
IF $\beta = 1$ THEN copy Y' to X
- b) IF $\alpha = 1$ THEN set X to 0
IF $\beta = 1$ THEN set X to X'

4 Show the hardware to implement the RTL statements developed for Problem 3.

6 Show the hardware to implement the following RTL code.

- a) $\alpha: X \leftarrow X + Y$
- b) $\beta: X \leftarrow X + Y' + 1$
- c) $\Gamma: X \leftarrow X \oplus Y$

7 Show the hardware to implement all three of the RTL statements of Problem 6 in one combined system. Control hardware ensures that no more than one of the control signals α , β , and Γ is active at any time.

II Show the hardware to implement the following micro-operations. X consists of four D flip-flops. Each micro-operation occurs when $\alpha = 1$.

- a) shl(X)
- b) shr(X)
- c) cil(X)
- d) cir(X)

Solutions:

1 Write valid RTL statements that realize the following transitions. All registers are 1-bit wide.

- a) IF $\alpha = 1$ THEN copy X to W and copy Z to Y .
- b) IF $\alpha = 1$ THEN copy X to W ; otherwise copy Z to Y .
- c) IF $\alpha = 0$ THEN copy X to W

a) $\alpha: W \leftarrow X, Y \leftarrow Z$

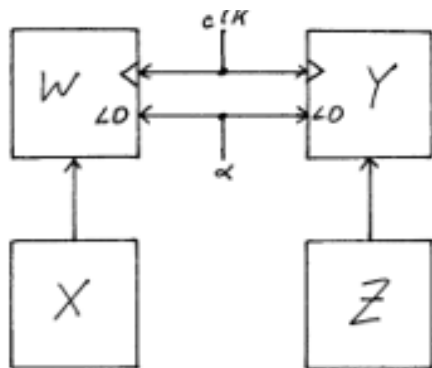
b) $\alpha: W \leftarrow X$

$\alpha': Y \leftarrow Z$

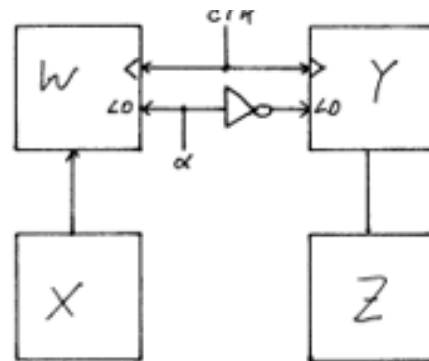
c) $\alpha': W \leftarrow X$

2 Show the hardware to implement the RTL statements developed for Problem 1.

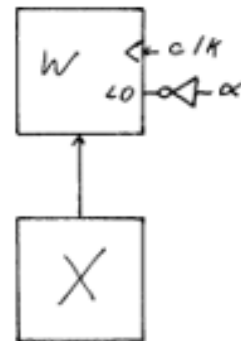
a)



b)



c)

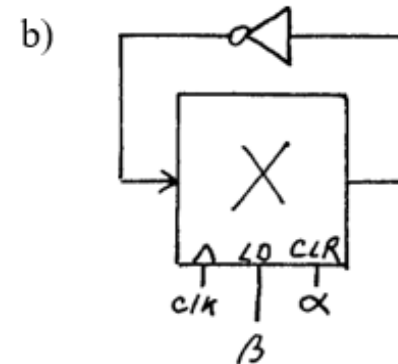
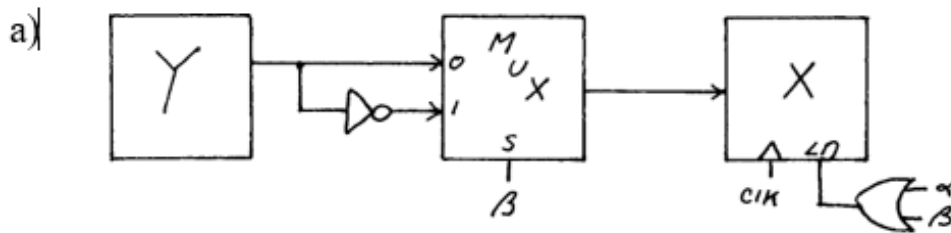


3 Write the RTL code for the following transitions. All registers are 1-bit wide. Signals α and β are never equal to 1 simultaneously.

- a) IF $\alpha = 1$ THEN copy Y to X
 IF $\beta = 1$ THEN copy Y' to X
- b) IF $\alpha = 1$ THEN set X to 0
 IF $\beta = 1$ THEN set X to X'

- a) $\alpha: X \leftarrow Y$
 $\beta: X \leftarrow Y'$
- b) $\alpha: X \leftarrow 0$
 $\beta: X \leftarrow X'$

4 Show the hardware to implement the RTL statements developed for Problem 3.



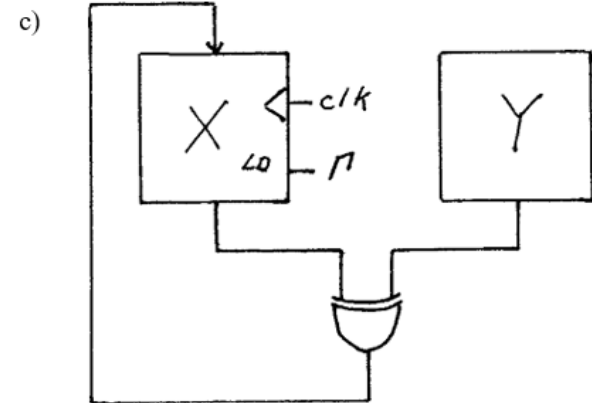
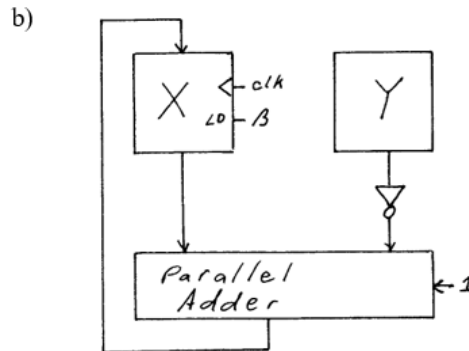
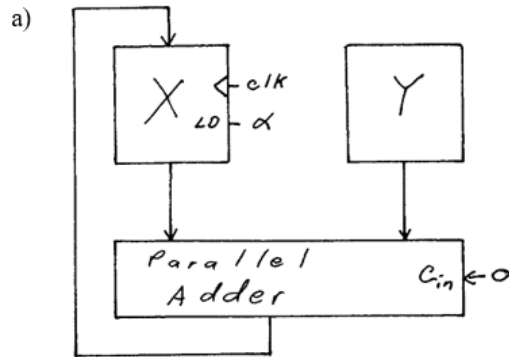
6

Show the hardware to implement the following RTL code.

a) $\alpha: X \leftarrow X + Y$

b) $\beta: X \leftarrow X + Y' + 1$

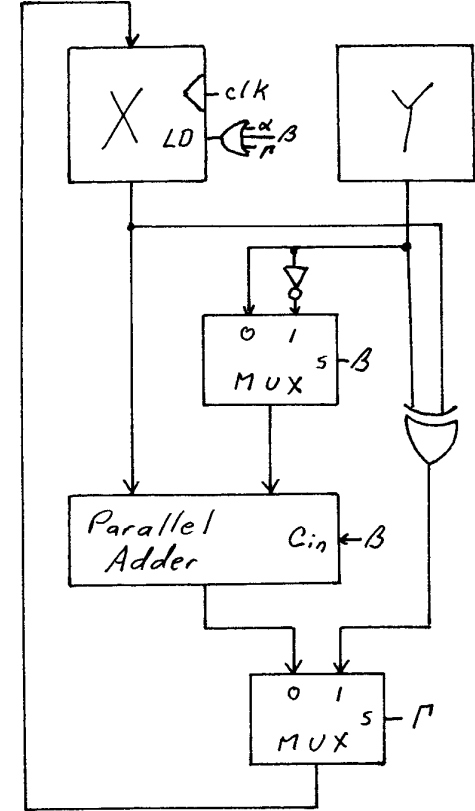
c) $\Gamma: X \leftarrow X \oplus Y$



6 Show the hardware to implement the following RTL code.

- a) $\alpha: X \leftarrow X + Y$
- b) $\beta: X \leftarrow X + Y' + 1$
- c) $\Gamma: X \leftarrow X \oplus Y$

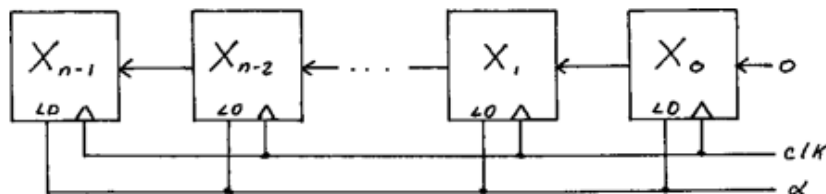
7 Show the hardware to implement all three of the RTL statements of Problem 6 in one combined system. Control hardware ensures that no more than one of the control signals α , β , and Γ is active at any time.



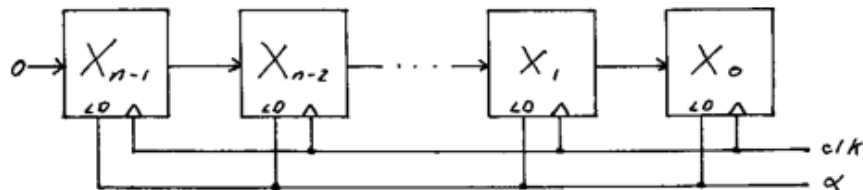
11 Show the hardware to implement the following micro-operations. X consists of four D flip-flops. Each micro-operation occurs when $\alpha = 1$.

- a) $\text{shl}(X)$
- b) $\text{shr}(X)$
- c) $\text{cil}(X)$
- d) $\text{cir}(X)$

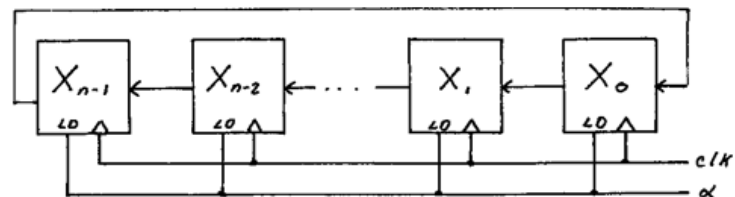
a)



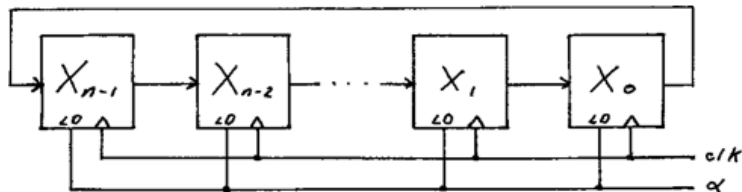
b)



c)



d)



Assignment:

1. Implement the given RTL codes into a single system using bus. . [PU 2017 Fall]
 $R1 \leftarrow R2, R3 \leftarrow R2$
 $R2 \leftarrow R3$
 $R3 \leftarrow R1, R2 \leftarrow R1$
2. Define RTL. Design a system that could implement the given RTL codes sing direct connection: [PU 2019 Spring]

R: $x \leftarrow x+y$
T: $x \leftarrow x-y$
L: $y \leftarrow y^x$

Assignment:

1. Uses of 8085 MP
2. Issues in 8085 MP

End of chapter 2